



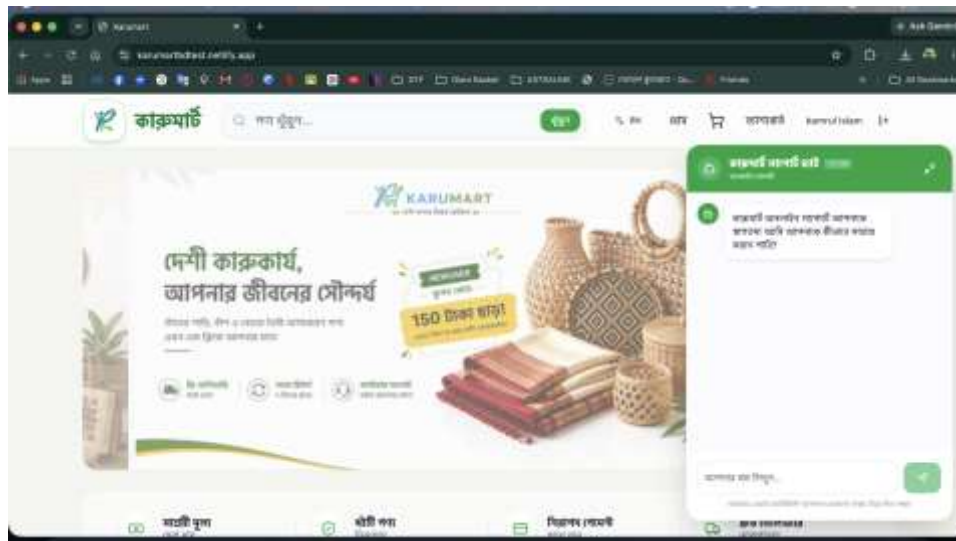
KARUMART
দেশী পণ্যের বিশ্বস্ত ঠিকানা
PROJECT REPORT

Field	Detail
Project Title	Startup: Karumart (দেশী পণ্যের বিশ্বস্ত ঠিকানা)
Course	CSE412 – Software Engineering
Institution	East West University
Semester	Summer 2026
Group	Group 16 — Team Pi (Section 3)
Field	Detail
Submitted By	Anik Saha — [REDACTED] (Team Lead) Saiba Arshi — [REDACTED] Kamrul Islam — [REDACTED] (section-02) Suraiya Afroz Momo — [REDACTED] Meherun Nesa — [REDACTED]
Submitted To	Dr. Mohammad Mahdi Hassan
Designation	Course Instructor, Department of Computer Science and Engineering
Submission Date	19 August 2026

Client Information

Client Name	Najmul Hasan Naim
Client Number	[REDACTED]
Client Address	Tangail,Dhaka

The Karumart Platform — Live Application



The Karumart home page, live at karumartbdtest.netlify.app, showing the bilingual storefront and the integrated AI support chat widget.

Declaration

We hereby declare that the project report titled "Karumart – Connecting Rural Artisans to the Digital Marketplace" is the result of our own original work, carried out under the CSE412 Software Engineering course at East West University during the Spring 2026 semester. All designs, source code, and documentation described herein were developed by our team (Group 16, Team Pi), except where explicitly cited.

Where information, requirements, or design guidance was supplied by the client, Mr. Najmul Hasan Naim, this has been acknowledged accordingly. Any external libraries, frameworks, or third-party services used in the implementation are referenced in the corresponding sections of this report.

This report has not been submitted, in whole or in part, for the award of any other degree, diploma, or similar title at this or any other institution.

Acknowledgement

We would like to express our sincere gratitude to our course instructor, Dr. Mohammad Mahdi Hassan, for continuous guidance, feedback, and encouragement throughout the development of Karumart. His direction on software engineering practice, requirement analysis, and project management was instrumental in shaping the outcome of this project.

We are equally thankful to our client, Mr. Najmul Hasan Naim, for sharing the original vision for a rural artisan marketplace, for participating in requirement-gathering interviews, and for providing feedback during our sprint review meetings, which allowed us to refine the platform through several iterations.

Finally, we thank the Department of Computer Science and Engineering at East West University for providing the academic environment and resources that made this project possible.

Abstract

Karumart is a full-stack, bilingual (Bengali/English) e-commerce web platform designed to connect rural artisans, weavers, and home-based craftspeople in Bangladesh with a wider base of consumers. Traditional producers of goods such as handloom textiles (jamdani, nokshi katha), pottery and earthenware, bamboo and cane crafts, jute goods, brass and bell-metal work, and other heritage products frequently lack the technical means or market access to sell independently online. Karumart addresses this gap by providing a managed, moderated marketplace in which sellers can list products under admin supervision, while consumers browse, search, and purchase through a familiar cart-and-checkout experience.

The platform is built with React 19, TypeScript, and Vite on the front end, styled with Tailwind CSS and shadcn/ui components, and served through a lightweight Express layer. Supabase provides the backend-as-a-service layer, supplying PostgreSQL data storage, authentication, file storage for product imagery, and real-time subscriptions, secured through Row Level Security policies. Google's Gemini AI is integrated on the seller side to auto-generate Bengali product descriptions, and on the consumer side to power a contextual shopping assistant chatbot.

The system defines four user roles — Consumer, Seller (internally modelled as "farmer"), Delivery Man, and Admin — each with a dedicated dashboard. Sellers must be approved by an administrator before their shop and products become publicly visible, and administrators retain the power to freeze shops that violate platform policy. Consumers can search in English, Bengali, or Banglish, apply vouchers at checkout, choose from Cash on Delivery or simulated digital payment methods (bKash, Nagad, card/SSLCommerz), track orders through a defined delivery lifecycle, download PDF invoices, and submit ratings, reviews, and complaints. Delivery personnel manage assigned consignments through a district-aware dashboard with live status updates.

This report documents the requirement analysis, system architecture, database design, implementation, and testing approach behind Karumart, drawing on the project's original client brief, sprint records, architecture wireframes, live application screenshots, and internal testing documentation. A directly observed terminal screenshot of the automated Vitest suite shows 146 executed tests across 10 files, with 143 passing (a 98.0% pass rate) and 3 known failures. The report also identifies current limitations — including simulated (non-live) payment gateways, the 3 outstanding test failures, and further integration testing that remains to be executed — and outlines realistic directions for future enhancement, such as genuine payment gateway integration and mobile applications.

Keywords: e-commerce, multi-vendor marketplace, rural artisans, React, TypeScript, Supabase, PostgreSQL, Gemini AI, Row Level Security, bilingual search.

Table of Contents

Declaration	2
Acknowledgement.....	2
Abstract.....	3
List of Figures.....	4
List of Tables.....	5
Chapter 1: Introduction.....	6
Chapter 2: Background Study / Literature Review	10
Chapter 3: Requirement Analysis.....	12
Chapter 4: System Analysis and Design	18
Chapter 5: Development and Implementation.....	25
Chapter 6: System Interface (Live Application Screenshots)	31
Chapter 7: Testing and Evaluation	44
Chapter 8: Results and Discussion	49
Chapter 9: Conclusion and Future Work.....	51
References	53
Appendix A: Important Code Snippets	54
Appendix B: Database Schema	55
Appendix C: Additional Design Artefacts	56
Appendix D: Test Cases	57

List of Figures

Figure 4.1: Three-Tier System Architecture of Karumart.....	19
Figure 4.2: Simplified Entity Relationship Diagram of Karumart.....	23
Figure 4.3: Use Case Diagram – Actors and Core Interactions	21
Figure 6.1: Karumart Home Page – Live Application	31
Figure 6.2: Search Results for "Printed Lungi"	32
Figure 6.3: Shopping Cart with Applied Item	32
Figure 6.4: Consumer Wishlist.....	33
Figure 6.5: Consumer Dashboard – Overview Tab.....	33
Figure 6.6: Order Tracking Modal	34
Figure 6.7: "Recommended for You" Product Grid	34
Figure 6.8: Admin Overview and Seller Management.....	35
Figure 6.9: Admin All Orders View.....	35
Figure 6.10: Voucher / Coupon Management	36

Figure 6.11: Flash Sale Management	36
Figure 6.12: Product Management (All Products)	36
Figure 6.13: Admin Activity Audit Log.....	37
Figure 6.14: Main Slider Banner Management	37
Figure 6.15: Side Promotional Banner Management	38
Figure 6.16: Bottom Static Banner Management	38
Figure 6.17: Administrator Account Management.....	38
Figure 6.18: Admin and Site Settings.....	39
Figure 6.19: Admin Overview – Analytics Charts	39
Figure 6.20: Admin Panel (Alternate / V2.0 Build) – All Orders	40
Figure 6.21: Seller Dashboard and Item Management Wireframe.....	40
Figure 6.22: Admin Dashboard Wireframe.....	41
Figure 6.23: Shop / Seller Storefront Wireframe	41
Figure 6.24: Karumart Laptop-Friendly Home Page	42
Figure 6.25: Karumart Mobile-Friendly Home Page	43
Figure 7.1: Actual Vitest Terminal Output from the Karumart Development Environment	44

List of Tables

Table 2.1: Traditional Approach vs Typical Online Marketplace vs Karumart.....	11
Table 3.1: Functional Requirements of Karumart.....	12
Table 3.2: Non-Functional Requirements	15
Table 3.3: Software Requirements / Technology Stack	16
Table 4.1: Database Entities and Purpose	21
Table 4.2: Representative Use Case Description – Place Order	20
Table 5.1: Development Environment	25
Table 7.0: Unit / Context-Level Test Suite — Reported Composition vs Directly Observed Run ...	45
Table 7.0b: Unit / Context-Level Test File Composition.....	45
Table 7.1: Functional Test Cases.....	46
Table 8.1: Achievement of Objectives	49

Chapter 1: Introduction

1.1 Introduction

Bangladesh has a long tradition of rural craftsmanship: handloom weaving, pottery, bamboo and cane work, jute goods, brass and bell-metal crafts, and traditional food and textile products are produced across villages throughout the country. Many of the people who make these products — often women, home-based workers, and small family workshops — possess deep craft skill but limited digital literacy, limited capital, and limited access to buyers outside their immediate area. At the same time, mainstream e-commerce platforms are generally designed around conventional retail and drop-shipping models, and are not tailored to the onboarding needs, verification requirements, or product categories of this specific seller group.

Karumart was conceived as a direct response to this gap: a dedicated, moderated online marketplace where rural artisans can list authentic, homemade, and traditional products, and where consumers can discover and purchase them with confidence. This chapter introduces the background, problem statement, objectives, and scope of the Karumart project, and explains how the remainder of this report is organized.

1.2 Background of the Project

Karumart was developed as the term project for the CSE412 (Software Engineering) course at East West University by Group 16 (Team Pi, Section 3), in direct collaboration with an external client, Mr. Najmul Hasan Naim. Unlike a purely academic exercise, the project began from a real client brief describing a genuine business need: a platform to sell traditional homemade clothing, pottery, handwoven mats, and similar goods produced by rural sellers who lacked the means to list on conventional e-commerce sites.

The client's original documentation specifically named categories such as traditional homemade clothing (jama kapor), pottery (matir jinis potro), and hand-woven mats (sitol pati) as target product types, and recommended a technology direction built around React/Vite for the frontend and Supabase for authentication, database, and real-time functionality. The development team used this brief, together with two rounds of structured client interviews and ongoing sprint review meetings, to iteratively design and build the platform documented in this report.

1.3 Problem Statement

Based on the client's requirements and the team's own analysis, the core problems addressed by Karumart are:

- Rural artisans and home-based producers create high-quality, authentic handmade goods but have no accessible, dedicated channel through which to reach buyers beyond their immediate locality.
- Mainstream e-commerce platforms are not designed with the technical limitations of non-technical rural sellers in mind, which creates a practical barrier to listing products independently.
- Buyers who are interested in authentic, locally sourced, traditional products lack a single trustworthy platform dedicated to this category, making discovery difficult and increasing the risk of encountering low-quality or misrepresented goods.

- There was no existing system, at the time of the client's request, that connected product creation, order management, quality moderation, and last-mile delivery specifically for this market segment.

1.4 Proposed Solution

Karumart addresses these problems through a managed, role-based marketplace architecture. Sellers register and submit shop details, but their storefront and product listings remain hidden from public view until an administrator reviews and approves them — this directly answers the trust and quality-control problem without requiring sellers to have any special technical skill beyond filling in a form. Every new product is assigned a unique, system-generated SKU automatically, and newly submitted products likewise wait in a moderation queue before they appear in customer search results.

On the consumer side, Karumart offers bilingual (Bengali/English) browsing and search — including Banglish (Romanized Bengali) query support — category and district-based filtering, a persistent cart with voucher support, and a full checkout-to-delivery-to-review order lifecycle. A dedicated delivery dashboard gives delivery personnel a simplified, mobile-friendly interface for updating order status in real time, which both the consumer and the admin can see reflected immediately. An administrator control panel ties the whole system together, giving platform operators the ability to approve or reject sellers and products, freeze shops that violate policy, manage promotional vouchers and banners, and review platform-wide statistics.

1.5 Aim of the Project

The aim of Karumart is to design and implement a secure, role-based, bilingual e-commerce platform that gives rural artisans and craftspeople a practical route to sell their authentic products online, while giving consumers a trustworthy, moderated shopping experience and giving platform administrators the tools needed to maintain product quality and marketplace integrity.

1.6 Objectives

- To develop a role-based authentication and authorization system supporting four distinct user types: Consumer, Seller, Delivery Man, and Admin.
- To provide sellers with a self-service dashboard for managing shop details, product listings, stock, and pricing, subject to administrator approval.
- To enable consumers to browse, search (in English, Bengali, and Banglish), filter by category and district, and purchase products through a persistent shopping cart and checkout flow.
- To implement an order lifecycle that tracks a purchase from placement through confirmation, pickup, transit, and delivery, including returns.
- To give administrators the tools to approve or reject sellers and products, freeze non-compliant shops, manage vouchers and promotional banners, and monitor platform activity through an audit log.
- To simplify seller onboarding by using AI-assisted (Gemini) generation of Bengali product descriptions from a product name and category.
- To improve customer support accessibility through an AI-powered chatbot capable of answering product- and order-related queries using live catalog data.

- To manage last-mile logistics through a dedicated delivery dashboard offering district-aware order assignment and real-time status updates.

1.7 Scope of the Project

The current implementation of Karumart covers the following:

- Consumer-facing storefront: home page with banners and flash sales, product browsing, product detail pages, seller storefronts, cart, checkout, order history, wishlist, and product reviews.
- Seller dashboard: shop profile management, product CRUD (create, read, update, delete) with AI-assisted descriptions, stock and price updates, shop status control (open / temporarily closed / permanently closed), and order fulfilment views.
- Delivery dashboard: district-filtered view of available and assigned orders, order status updates, and printable delivery slips.
- Admin panel: seller and product approval queues, order oversight, complaint handling, voucher and flash-sale management, banner management, admin action logging, badge assignment, and platform settings.
- Supporting features: bilingual interface (Bengali/English), multilingual and Banglish product search, PDF invoice and delivery-slip generation, and a Gemini-powered AI shopping assistant.

The following are explicitly outside the current scope of the implemented system:

- Integration with live third-party payment gateways. bKash, Nagad, and card/SSLCommerz payment options exist in the checkout flow, but the corresponding payment service currently simulates gateway responses rather than connecting to a real payment processor.
- Native mobile applications; Karumart is currently a responsive web application only.
- Advanced analytics, recommendation engines, or machine-learning-driven personalization beyond the descriptive statistics shown on the admin dashboard.

1.8 Significance of the Project

For rural sellers, Karumart provides a low-barrier entry point into digital commerce: a seller does not need any technical background to register a shop, list a product, or manage orders, and the admin approval step gives their storefront credibility that an unmoderated listing would lack. For consumers, the platform offers a dedicated destination for authentic, traditional, and handmade goods, with the trust signals (approval status, shop badges, ratings and reviews, complaint handling) that this category of purchase particularly benefits from. For the platform operator (the client), Karumart provides direct oversight of seller quality and platform integrity through the admin approval and shop-freezing mechanisms, without requiring manual, ad-hoc verification outside the system.

1.9 Limitations

Based on the current implementation, the following limitations can reasonably be identified:

- Digital payments (bKash, Nagad, card) are simulated in the codebase rather than connected to a live payment gateway; only Cash on Delivery reflects an unsimulated real-world payment flow.

- The Gemini AI API key used for product-description generation and the shopping chatbot is read through a client-exposed environment variable (VITE_GEMINI_API_KEY) in the current build, rather than being proxied through a private backend endpoint.
- Stock decrements at checkout are performed as a sequence of client-driven database calls rather than as a single atomic database transaction or stored procedure, which introduces a theoretical race-condition risk under concurrent purchases of the same low-stock item.
- An actual test run of the automated unit/context suite shows 3 failing tests across 2 files (out of 146 total), which remain to be diagnosed and fixed. The separately documented integration test plan (twenty test cases spanning authentication, cart, checkout, payment, search, and admin flows) is, at the time of this report, defined but not yet marked as executed; results are recorded as "to be verified" pending a dedicated test run.
- The project does not currently include a native mobile application.

1.10 Organization of the Report

The remainder of this report is organized as follows. Chapter 2 discusses the background domain and situates Karumart relative to traditional and existing online approaches. Chapter 3 presents the requirement analysis, including functional and non-functional requirements and a feasibility study. Chapter 4 covers system analysis and design, including the architecture, use cases, database design, and UI design principles. Chapter 5 describes the development and implementation of the system in detail, module by module. Chapter 6 presents the system's interface through the project's design wireframes. Chapter 7 covers the testing strategy and results reported for the project. Chapter 8 discusses the results achieved against the stated objectives. Chapter 9 concludes the report and outlines directions for future work, followed by references and appendices.

Chapter 2: Background Study / Literature Review

2.1 Introduction

Karumart operates in the intersection of two domains: rural craft and cottage industry commerce, and multi-vendor e-commerce marketplace platforms. This chapter briefly reviews how transactions in this domain are typically handled without a dedicated system, discusses the general characteristics of online marketplace platforms, and positions Karumart's approach relative to both.

2.2 Existing System / Traditional Approach

In the traditional approach, rural artisans and craftspeople sell their goods through local markets (haats and bazaars), door-to-door sales, or through intermediaries and wholesalers who purchase goods at a discount and resell them in urban markets. This approach has several structural drawbacks: producers rarely capture the full retail value of their work because intermediaries take a margin, market reach is limited to whoever physically visits the local market or is reached by the intermediary's own distribution network, and there is no persistent record of a producer's catalog, pricing, or reputation that a distant buyer could review before purchasing.

2.3 Existing Online Solutions

General-purpose e-commerce platforms and social-commerce (selling through Facebook pages or messaging apps) are the most common alternative currently available to small Bangladeshi sellers. General marketplace platforms typically support a wide range of product categories and seller types, with varying degrees of seller verification and varying levels of interface simplicity. Social-commerce, conducted informally over Facebook or WhatsApp, is popular precisely because it requires no technical setup, but it also provides no structured cart, checkout, order tracking, or seller verification — every transaction is negotiated manually.

2.4 Problems with Existing Approaches

- Manual processing: informal or intermediary-based sales rely on manual price negotiation, manual record-keeping, and manual delivery arrangement, which does not scale beyond a small, local customer base.
- Accessibility: rural producers frequently lack the technical literacy, and general e-commerce platforms lack the guided onboarding, needed for a non-technical seller to list products independently.
- Information management: without a structured product catalog, buyers cannot reliably compare prices, check stock availability, or view a seller's full range of products.
- Search difficulty: informal channels (e.g. scrolling a Facebook page) provide no structured search or filtering, making it hard for a buyer to find a specific traditional product by name, category, or region.
- Order and process tracking: manual sales processes typically provide no formal order status, tracking, invoice, or return mechanism.

2.5 Proposed Karumart System

Karumart addresses the problems above by replacing informal, intermediary-mediated exchange with a structured, moderated digital marketplace. Sellers list products through a guided dashboard rather than

negotiating manually; every listing carries a stable price, stock count, and category; every order generates a trackable status history, a digital invoice, and (where applicable) a return-request pathway; and every seller and product passes through an administrator review step before appearing publicly, giving buyers a level of quality assurance that informal channels do not offer.

2.6 Comparative Analysis

Table 2.1: Traditional Approach vs Typical Online Marketplace vs Karumart

Feature	Traditional / Informal Approach	Typical General E-Commerce Platform	Karumart
Seller onboarding	No formal process; word of mouth	Self-service, often with minimal vetting	Self-service registration, but shop remains hidden until admin approval
Product listing	No structured catalog	Structured catalog, open to most product types	Structured catalog with automatic SKU generation and mandatory approval
Search & discovery	Manual / word of mouth	Keyword search, typically English-first	English, Bengali, and Banglish search with synonym matching
Order tracking	None	Order status tracking, generally supported	Defined status lifecycle: pending, confirmed, picked up, on the way, delivered, plus returns
Quality control	Buyer must judge seller directly	Varies by platform; often limited for small platforms	Admin approval of sellers and products; shop-freeze mechanism for violations
Payments	Cash in person	Live payment gateways generally supported	Cash on Delivery live; bKash / Nagad / card currently simulated
Delivery coordination	Ad hoc	Varies; often outsourced to courier partners	Dedicated delivery dashboard with district-aware assignment and real-time status

2.7 Chapter Summary

This chapter situated Karumart relative to the informal, intermediary-based sales channels traditionally available to rural artisans, and relative to general-purpose online marketplaces. The comparison shows that Karumart's principal points of differentiation are its admin-moderated seller and product approval workflow, its multilingual search tailored to Bengali-speaking users, and its dedicated delivery-tracking dashboard, all built around the specific needs of the rural handicraft and homemade-goods category rather than being a generic online store.

Chapter 3: Requirement Analysis

3.1 Introduction

Requirement analysis translates the client's business goals into a concrete specification that the engineering team can design and build against. For Karumart, requirements were gathered primarily through structured client interviews with Mr. Najmul Hasan Naim, supplemented by an internally authored Project Requirement and User Story document, and refined iteratively across ten sprints as the team's own technical understanding of the problem matured. This chapter presents the resulting functional and non-functional requirements, hardware and software requirements, and a feasibility assessment.

3.2 Stakeholders / User Types

Karumart is built around four primary user roles, matching the roles defined in the platform's database schema (the `profiles.role` field) and the client's original requirement documentation:

- Consumer — the general buyer who registers, browses, and purchases products.
- Seller (stored internally as "farmer" in the schema and code, and labelled "বিক্রেতা" / Seller in the interface) — a rural artisan or craftsperson who lists and manages products under a shop profile.
- Delivery Man — logistics personnel responsible for picking up and delivering orders.
- Admin — the platform operator with full oversight of sellers, products, orders, vouchers, and platform settings.

3.3 Functional Requirements

Functional requirements are grouped below by module and numbered for traceability.

User Management

Table 3.1: Functional Requirements of Karumart

ID	Requirement	Actor	Description
FR-01	Self-registration	Consumer, Seller, Delivery Man	A visitor can create an account by choosing a role, email, password, and full name; seller accounts are created with an initial 'pending' status.
FR-02	Login	All roles	A registered user can log in with email and password; administrators use a dedicated admin login page.
FR-03	Session persistence	All roles	The system keeps the user logged in across page reloads using a Supabase authentication session, and restores the user's profile and role on load.
FR-04	Role-based route protection	All roles	Certain routes (e.g. /checkout, /dashboard) are only reachable by

ID	Requirement	Actor	Description
			authenticated users, and some are further restricted by role.
FR-05	Profile / shop management	Seller	A seller can update shop name, description, address, logo, and cover image.

Product / Catalog Management

ID	Requirement	Actor	Description
FR-06	Add product	Seller	A seller can add a new product with name, category, price, quantity, unit, description, and image; the system automatically generates a unique SKU.
FR-07	AI-assisted description	Seller	A seller can request an automatically generated Bengali product description from the Gemini AI service, based on product name and category.
FR-08	Edit / update product	Seller	A seller can update price, stock quantity, category, description, and SKU of an existing product.
FR-09	Delete product	Seller	A seller can remove a product from their catalog.
FR-10	Product moderation	Admin	Newly added or edited products remain unapproved until an administrator reviews and approves them; unapproved products do not appear in consumer search or listings.
FR-11	Browse and filter products	Consumer	A consumer can browse all approved products, filter by category, and filter by district/region.
FR-12	Search products	Consumer	A consumer can search products by keyword in English, Bengali, or Banglish, matching product name, category, shop name, and seller address.
FR-13	View product detail	Consumer	A consumer can view full product details, including price, stock status, description, seller information, and existing reviews.

ID	Requirement	Actor	Description
FR-14	Flash sale support	Seller / Admin	Products can be marked with an active flash-sale price and countdown, displayed on the home page.

Cart, Checkout, and Order Module

ID	Requirement	Actor	Description
FR-15	Add to cart	Consumer	A consumer can add a product to a persistent (browser-stored) shopping cart.
FR-16	Update / remove cart item	Consumer	A consumer can increase, decrease, or remove the quantity of an item already in the cart.
FR-17	Apply voucher	Consumer	A consumer can enter a voucher code at checkout; the system validates the code's active status, expiry date, and minimum purchase requirement before applying a percentage or fixed discount.
FR-18	Checkout	Consumer	An authenticated consumer can submit shipping details, choose a payment method (Cash on Delivery, bKash, Nagad, or card), and place an order.
FR-19	Stock and shop validation at checkout	System	Before an order is created, the system verifies that each item is still in sufficient stock and that the seller's shop is open and not frozen.
FR-20	Order status tracking	Consumer, Delivery Man, Admin	An order progresses through a defined status lifecycle (pending, confirmed, picked up, on the way, delivered, cancelled, or a return sequence), visible to the consumer, the assigned delivery person, and the admin.
FR-21	Invoice generation	Consumer	A consumer can view and download a PDF invoice for a completed order.
FR-22	Return request	Consumer	A consumer can submit a complaint / return request (refund, replacement, or quality issue) against a specific order.
FR-23	Rating and review	Consumer	A consumer can submit a star rating and comment for a product after an order is delivered.

Delivery Module

ID	Requirement	Actor	Description
FR-24	View assigned / available orders	Delivery Man	A delivery person can view orders assigned to them, filtered by pickup or delivery district.
FR-25	Update delivery status	Delivery Man	A delivery person can update an order's status through the delivery lifecycle in real time.
FR-26	Print delivery slip	Delivery Man	A delivery person can generate and download a PDF delivery / packing slip for an order.

Administrative Functions

ID	Requirement	Actor	Description
FR-27	Approve / reject sellers	Admin	An admin can review pending seller registrations and approve or reject them.
FR-28	Approve / reject products	Admin	An admin can review newly submitted or edited products and approve or reject them before they go live.
FR-29	Freeze / unfreeze shop	Admin	An admin can freeze a shop that violates platform policy, immediately hiding its products from consumers and blocking the seller's dashboard actions.
FR-30	Manage vouchers	Admin	An admin can create, activate, deactivate, and view usage of promotional voucher codes.
FR-31	Manage banners	Admin	An admin can manage homepage slider banners, side promo banners, and bottom static banners.
FR-32	Assign badges	Admin	An admin can toggle trust badges (Best Seller, Official Store, Karu Mall) on a seller's profile.
FR-33	View platform statistics	Admin	An admin can view order counts by status, revenue figures, sales trends, top products, and category breakdowns on a statistics dashboard.
FR-34	Manage complaints	Admin	An admin can view and resolve consumer complaints and return requests.

ID	Requirement	Actor	Description
FR-35	Audit logging	Admin	Administrative actions (approvals, freezes, badge changes, etc.) are recorded in an admin action log for accountability.
FR-36	Manage additional admins	Admin	An existing admin can create new administrator accounts.
FR-37	Site settings	Admin	An admin can configure platform-wide settings such as the delivery charge, free-delivery threshold, and support contact details.

3.4 Non-Functional Requirements

Table 3.2: Non-Functional Requirements

Category	Requirement	Status
Usability	The interface should be usable by non-technical sellers with minimal training, and should support both Bengali and English.	Implemented — bilingual UI with a persistent language switcher
Performance	Product listing and search should return results without perceptible delay under normal load.	Implemented for current data volumes; not load-tested at scale
Reliability	Core purchase flows (cart, checkout, order creation) should behave consistently and report clear errors on failure.	Implemented, with client-side error handling and toast notifications
Security	User passwords must never be stored in plain text; access to data must be restricted according to user role.	Implemented via Supabase Auth (hashed credentials) and PostgreSQL Row Level Security policies
Maintainability	The codebase should be organized into clear, reusable modules (contexts, services, pages, components) to support future extension.	Implemented — TypeScript, React Context providers, and a dedicated services layer
Compatibility	The application should function correctly on current versions of major desktop and mobile browsers.	Recommended target; not documented as formally cross-browser tested
Scalability	The system should be able to accommodate a growing number of sellers, products, and orders without redesign.	Supported architecturally through Supabase's managed PostgreSQL backend; not load-tested

Category	Requirement	Status
Responsiveness	The interface should adapt to desktop, tablet, and mobile screen sizes.	Implemented using Tailwind CSS responsive utility classes

The distinction above is important: usability, security, maintainability, and responsiveness reflect characteristics that are directly observable in the implemented codebase. Performance, compatibility, and scalability are treated as recommended quality targets consistent with the chosen architecture, since no dedicated load-testing or cross-browser test report was available for this project.

3.5 Hardware Requirements

As a web application, Karumart itself does not impose unusual hardware demands. For development, a standard laptop or desktop computer capable of running Node.js and a modern web browser is sufficient. For end users, any device with a modern web browser and an internet connection (desktop, laptop, tablet, or smartphone) can access the platform, since no native client installation is required.

3.6 Software Requirements

Table 3.3: Software Requirements / Technology Stack

Category	Technology
Programming language	TypeScript (application code), SQL (database schema)
Frontend framework/library	React 19
Build tool / dev server	Vite 6
Routing	React Router DOM 7
Styling	Tailwind CSS 4, shadcn/ui component primitives
Backend as a Service	Supabase (PostgreSQL database, Authentication, Storage, Realtime, Row Level Security)
Server runtime	Node.js with Express (serves the Vite build / dev middleware)
AI service	Google Gemini API (@google/genai), model gemini-flash-latest
PDF generation	jsPDF and html2canvas (client-side invoice / delivery-slip rendering)
Charting	Recharts (admin analytics)
Notifications	Sonner and react-hot-toast
IDE / editor	Visual Studio Code (typical for the development environment; not separately documented)
Browser	Any modern Chromium-, Gecko-, or WebKit-based browser
Version control	Git

3.7 Feasibility Study

3.7.1 Technical Feasibility

The technologies selected — React, TypeScript, Vite, and Supabase — are mature, well-documented, and widely used in industry, which made them appropriate for a team operating under an academic semester timeline. Supabase in particular reduced the backend engineering burden considerably, since authentication, a managed PostgreSQL database, file storage, and real-time subscriptions were available out of the box rather than needing to be built from scratch. The Gemini AI integration required only a straightforward SDK call rather than any custom machine-learning infrastructure. On this basis, the project was technically feasible for a five-person student team within a single semester.

3.7.2 Operational Feasibility

The client's core operational need — a self-service storefront for non-technical rural sellers, with administrator oversight — maps directly onto the implemented seller-registration-and-approval workflow and the admin moderation dashboard. Sellers interact with a guided form-based interface rather than needing to understand databases or code, and administrators are given a single dashboard from which to manage the entire moderation workload, which keeps day-to-day operational overhead manageable.

3.7.3 Economic Feasibility

Development costs were limited to the team's own time, consistent with the project's academic context. Ongoing operational costs are modest: Supabase and typical static-hosting providers offer usable free or low-cost tiers suitable for an early-stage platform, and the Gemini API is billed per request at a low per-call cost. The most significant future cost driver would be integrating and maintaining live payment gateway connections (bKash, Nagad, SSLCommerz), which is currently deferred.

3.7.4 Schedule Feasibility

The project was executed across multiple two-week sprints (documented from Sprint 3 through Sprint 7 in the team's sprint tracking sheets), with tasks distributed across the five team members and reviewed in regular client meetings. Sprint records show a consistent pattern of carrying incomplete tasks forward and completing them in the following sprint, indicating that the schedule, while occasionally tight, was ultimately workable within the semester timeline.

3.8 Chapter Summary

This chapter defined the four Karumart user roles and translated the client's requirements into thirty-seven numbered functional requirements across user management, catalog management, cart/checkout/orders, delivery, and administration, along with a set of non-functional requirements distinguishing implemented characteristics from recommended targets. The feasibility study concluded that the project was technically, operationally, economically, and organizationally feasible within an academic semester, given the chosen Supabase-centric architecture.

Chapter 4: System Analysis and Design

4.1 Introduction

This chapter presents the architectural and design view of Karumart: the overall system architecture, the request/data workflow, the system's actors and use cases, the database design, the user interface design approach, and the security controls built into the platform. The design descriptions in this chapter are drawn directly from the implemented source code (React component structure, Supabase client usage, and the SQL schema) rather than from an idealized or generic e-commerce design.

4.2 System Architecture

Karumart follows a three-tier architecture in which a React single-page application (the presentation tier) communicates directly with Supabase, a managed backend-as-a-service platform (the backend/data tier), rather than through a custom-built REST or GraphQL API server. A thin Node.js/Express layer is present only to serve the built application and Vite's development middleware; it does not implement business-logic routes of its own. A separate AI service call path connects the frontend directly to the Google Gemini API for description generation and the shopping assistant chatbot.

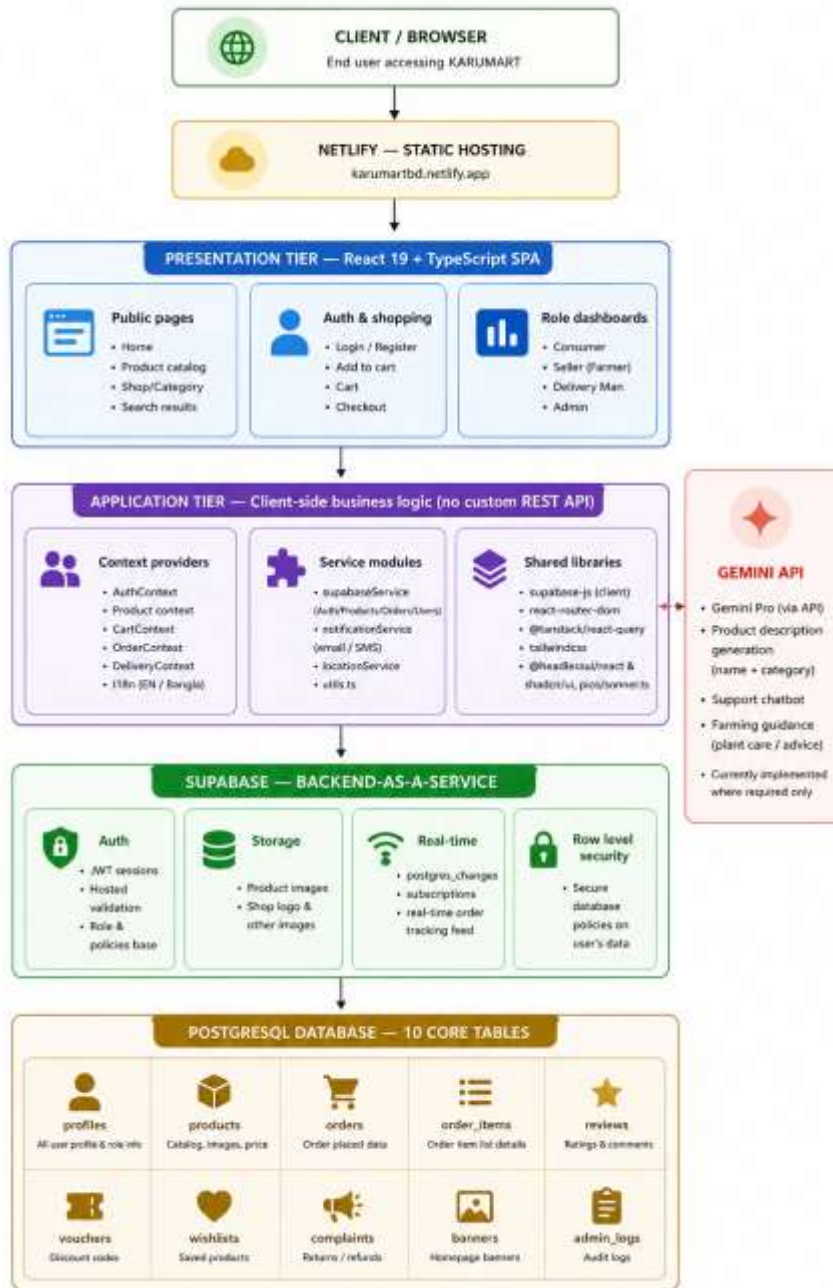


Figure 4.1: Three-Tier System Architecture of Karumart

This architecture differs from a classical MVC web application in one important respect: because Supabase's client library is called directly from React components and context providers, the "application tier" in Karumart is realized as a set of React Context providers (AuthContext, CartContext, LanguageContext) and TypeScript service modules (paymentService.ts, adminService.ts, geminiService.ts) rather than as server-side controllers. Authorization is enforced partly on the client (through route guarding) and partly at the database layer (through PostgreSQL Row Level Security policies attached directly to Supabase tables).

4.3 System Workflow

A typical consumer purchase workflow proceeds as follows:

- 1. The consumer browses the home page or product listing, optionally filtering by category, district, or search keyword.
- 2. The consumer opens a product detail page and adds the item to their cart; cart state is held in the CartContext provider and persisted to the browser's local storage.
- 3. The consumer proceeds to checkout, which requires an authenticated session; unauthenticated users are redirected to the login page.
- 4. At checkout, the system re-validates stock levels and the seller's shop status (open and not frozen) for every cart item before allowing the order to proceed.
- 5. An order record is created in Supabase with status 'pending', followed by the corresponding order_items records.
- 6. If a digital payment method was selected, the PaymentService module simulates the relevant gateway call and returns a transaction ID, which is written back to the order; for Cash on Delivery, no payment step is required at this stage.
- 7. Product stock quantities are decremented, the cart is cleared, and the consumer is redirected to their dashboard, where the new order appears in their order history.
- 8. As the seller and delivery personnel update the order's status, the consumer's order history and the admin dashboard reflect the change through Supabase queries (and, for site-wide settings, through Supabase real-time subscriptions).

4.4 Use Case Analysis

Karumart's four actors — Consumer, Seller, Delivery Man, and Admin — interact with the system as summarized below.

Table 4.2: Representative Use Case Description – Place Order

Field	Description
Use Case	Place Order
Actor	Consumer (authenticated)
Preconditions	The consumer is logged in and has at least one item in their cart.
Main Flow	Consumer reviews cart totals, enters shipping address and phone, optionally applies a voucher, selects a payment method, and submits the order.
Alternative Flow	If any item's stock is insufficient, or the seller's shop is closed or frozen, the system rejects the order with an explanatory message before any order record is created. If an applied voucher is expired or the order does not meet its minimum purchase requirement, the voucher is rejected.
Postconditions	An order and its order_items are created in the database, product stock is decremented, the cart is cleared, and the consumer is redirected to their dashboard.

The diagram below, based on the project's finalized use-case diagram, summarizes the principal use cases per actor. Register, Login, Logout, and Update Profile are common to all account-holding actors;

Manage Product (Add/Edit/Delete Product), View Orders, Update Order Status, and Track Order belong to the Seller; Browse Product, Manage Cart (Add to Cart/Remove Item/Update Quantity), Place Order (Select Address/Make Payment/Confirm Order/Cash on Delivery), Track Order, and View Order History belong to the Consumer; View Assigned Orders, Update Delivery Status, and Confirm Delivery belong to the Delivery Person; and Verify Seller, Approve Product, Manage Users (Activate/Suspend/Delete User), Monitor Orders, and View Reports belong to the Admin.

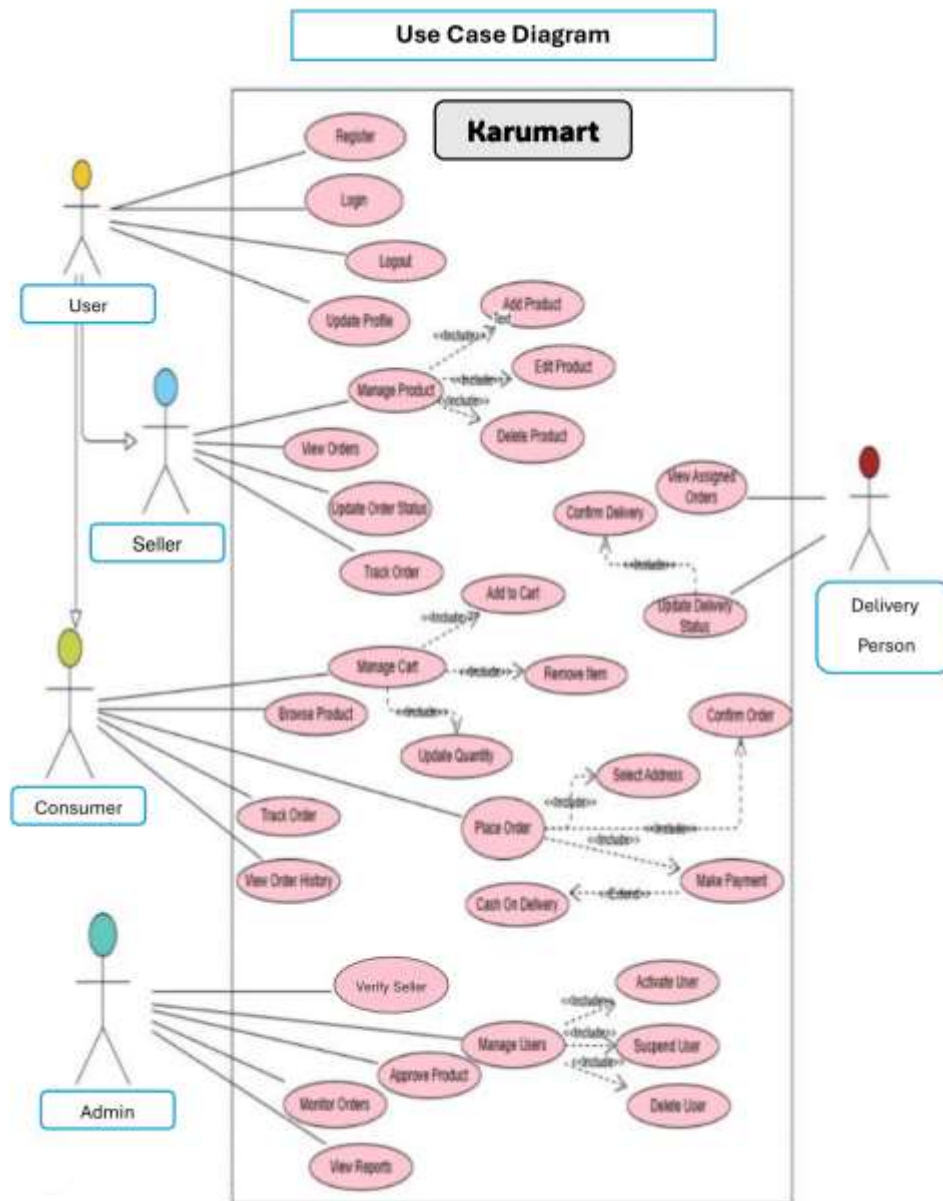


Figure 4.3: Use Case Diagram – Actors and Core Interactions (Karumart project use case diagram)

4.5 Data Flow Diagram

At the context level, Karumart can be described as a single process — the Karumart web application — that receives requests from four external entities (Consumer, Seller, Delivery Man, and Admin) and reads from or writes to a single logical data store realized as the Supabase PostgreSQL database, with a

secondary external interface to the Google Gemini API for AI-generated text. At Level 1, this process decomposes into the major functional flows already described in Section 3.3: authentication, catalog management, cart and checkout, order and delivery management, and administration, each reading and writing its own subset of the database tables described in Section 4.6. These diagrams are described here in narrative and tabular form; for formal presentation, they should be redrawn as standard DFD notation (using drawing software such as Visio, Lucidchart, or the Word drawing tools) rather than relying on this textual description alone.

4.6 Database Design

Karumart's data is stored in a PostgreSQL database managed through Supabase. The project's `database_setup.sql` script defines the initial ten tables and baseline Row Level Security policies; several additional columns referenced by the application's TypeScript types and queries (for example, `cover_url` and the flash-sale fields on products, and the `delivery_charge` / `free_delivery_threshold` fields on `site_settings`) indicate that the schema was extended iteratively beyond this initial script over the course of development, consistent with the "Supabase Database Schema Update" tasks recorded in the project's sprint logs.

Table 4.1: Database Entities and Purpose

Entity / Table	Important Attributes	Purpose
profiles	id (PK, = auth.users id), role, email, full_name, shop_name, shop_status, status, is_frozen, badge_best_seller, badge_official, badge_krishi_mall	Stores every account (consumer, seller, delivery man, admin) and, for sellers, their shop information and moderation status.
products	id (PK), farmer_id (FK → profiles), name, category, price, quantity, unit, sku, image_url, is_approved	Stores the product catalog; <code>is_approved</code> gates public visibility.
orders	id (PK), consumer_id (FK), delivery_man_id (FK), status, payment_method, payment_status, transaction_id, total_amount, address, phone	Master record of a purchase, its payment, and its delivery status.
order_items	id (PK), order_id (FK), product_id (FK), quantity, price_at_time	Line items belonging to an order, preserving the price at the time of purchase.
reviews	id (PK), product_id (FK), consumer_id (FK), rating, comment	Consumer ratings and comments on purchased products.
vouchers	id (PK), code, discount_type, value, min_purchase, expiry_date, is_active	Promotional discount codes applied at checkout.

Entity / Table	Important Attributes	Purpose
complaints	id (PK), user_id (FK), order_id (FK), subject/reason, status	Consumer-submitted returns, refund requests, and quality complaints.
banners / static_banners	id (PK), image_url, title, link_url	Homepage promotional slider and static banner content managed by the admin.
admin_logs	id (PK), admin_id (FK), action, target_id, details	Audit trail of administrative actions such as approvals and freezes.
site_settings	id (PK = 'site_config'), hotline, email, delivery_charge, free_delivery_threshold	Single-row table of platform-wide configuration values.

4.7 Entity Relationship Diagram

The simplified entity relationship diagram below reflects the tables and foreign-key relationships actually defined in database_setup.sql and referenced throughout the application code. profiles acts as the central entity, since every product, order, review, wishlist entry, complaint, and admin log ultimately references a profile through a foreign key.

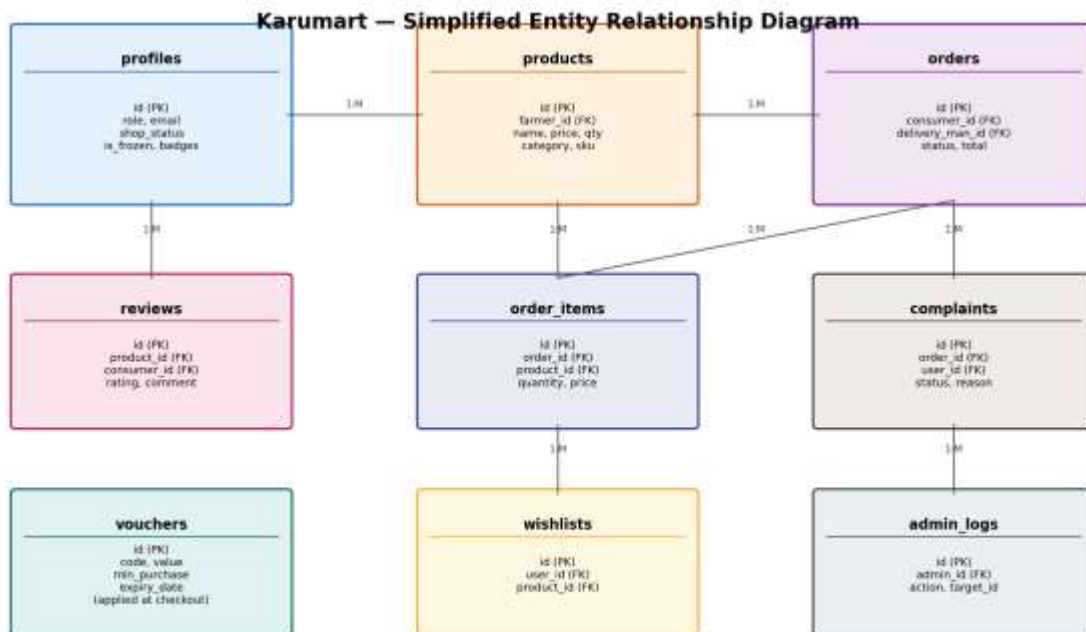


Figure 4.2: Simplified Entity Relationship Diagram of Karumart

4.8 User Interface Design

The interface follows a conventional storefront layout: a persistent top navigation bar (logo, search, language switch, cart, and login/account menu), a hero banner carousel, trust-badge strip, promotional and flash-sale sections, category browsing, and a paginated product grid, all visible in the project's own

architecture wireframes (Chapter 6). Product cards consistently show image, name, price (with strikethrough original price and discount percentage where applicable), and an Add to Cart control. Forms throughout the application (registration, login, checkout, product creation) follow a consistent labelled-input pattern built on shadcn/ui form primitives (Input, Label, Select, Textarea).

Navigation and layout are implemented responsively using Tailwind CSS utility breakpoints, and the interface is fully bilingual: every visible label is drawn from a central translation dictionary (`src/lib/translations.ts`) keyed by a language context, so switching between Bengali and English updates the entire interface — including role names, order statuses, and category names — consistently, without a page reload.

4.9 Security Design

The following security controls are present in the current implementation:

- Authentication: user sign-up and login are handled by Supabase Auth, which stores credentials as salted hashes rather than plaintext and issues JWT-based session tokens.
- Authorization / Row Level Security: PostgreSQL Row Level Security is enabled on every application table, with explicit policies (for example, public read access to products and profiles, and authenticated-only insert access to orders) defined in `database_setup.sql`.
- Client-side route protection: the `ProtectedRoute` component redirects unauthenticated users to the login page and can additionally restrict a route to a specific list of allowed roles (used, for example, to restrict `/checkout` to the consumer role).
- Seller/shop enforcement: the checkout flow explicitly re-checks a product's seller's `shop_status` and `is_frozen` flag on the server-stored data before allowing an order, so a frozen or closed shop cannot receive new orders even if a consumer already has its product in their cart.
- Moderation gate: unapproved products (`is_approved = false`) and unapproved sellers are excluded from public queries, limiting exposure of unvetted content.

One security-relevant limitation should also be noted for accuracy: the Gemini AI API key is read in the browser through a Vite public environment variable (`VITE_GEMINI_API_KEY`), which means it is bundled into client-side JavaScript rather than being isolated on a private backend, as would be recommended for production-grade key handling. Proxying Gemini calls through a private backend endpoint is identified as a recommended future improvement in Chapter 9.

4.10 Chapter Summary

This chapter presented Karumart's three-tier architecture built around a React frontend and a Supabase backend-as-a-service, its core purchase workflow, its four-actor use case structure, its ten-table PostgreSQL schema and entity relationships, its bilingual and responsive UI design approach, and the authentication, authorization, and moderation controls that secure the platform. The following chapter describes how this design was implemented in practice.

Chapter 5: Development and Implementation

5.1 Introduction

This chapter describes how the design presented in Chapter 4 was implemented: the development environment and technology stack, the project's folder structure, the frontend and data-access implementation, authentication and authorization, and the implementation of each major functional module. A small number of representative code excerpts are included, each followed by an explanation of its role in the system.

5.2 Development Environment

Table 5.1: Development Environment

Component	Technology / Tool	Purpose
Package manager / runtime	Node.js, npm	Dependency management and script execution
Build tool	Vite 6.2	Development server with hot module reload and production bundling
Language	TypeScript 5.8	Static typing across components, contexts, services, and data models
Local server	Express 4.21 (server.ts)	Wraps Vite's middleware in development and serves the static build in production
Backend platform	Supabase (hosted)	Managed PostgreSQL database, authentication, storage, and real-time channel
Version control	Git	Source control (project history maintained in a local Git repository)

5.3 Technology Stack

React 19 with TypeScript

React provides the component-based architecture for the entire frontend. Application state that must be shared across many components — the logged-in user, cart contents, and the active language — is managed through three React Context providers (AuthContext, CartContext, LanguageContext) rather than a separate state-management library, which keeps the dependency footprint small while still avoiding prop-drilling across deeply nested dashboard components. TypeScript is used throughout, with a central types/index.ts file defining the shape of every domain object (Profile, Product, Order, Voucher, Complaint, Review, and so on), which keeps the data contracts between the database, the service layer, and the UI consistent.

Vite and Express

Vite is used both as the development server (offering fast hot-module-reload while building the many dashboard pages) and as the production bundler. The accompanying server.ts file wraps Vite's dev middleware in development and, in production, serves the compiled dist/ folder as static files through

Express, with a catch-all route that returns `index.html` so that client-side routing (via React Router) continues to work correctly on a full page refresh.

Tailwind CSS and shadcn/ui

Styling is implemented with Tailwind CSS's utility-class approach, avoiding hand-written CSS files for most components. Structural UI primitives — buttons, cards, dialogs, dropdown menus, inputs, selects, tabs, and tables — are built on shadcn/ui, a library of accessible, unstyled component patterns that the project has copied into `src/components/ui/` and customized with Tailwind classes, rather than being pulled in as an opaque third-party dependency.

Supabase

Supabase is the single backend dependency for persistence, authentication, and file storage. The client is initialized once in `src/lib/supabase.ts`, using a lazy proxy pattern that only constructs the actual Supabase client on first use and throws a clear configuration error if the required environment variables are missing. Every data-fetching call in the application — from the product listing on the home page to the admin's revenue statistics — goes through this single client instance, calling Supabase's PostgREST-based query interface directly from React components and context providers.

Google Gemini AI

The `@google/genai` SDK connects the frontend to Google's Gemini API (model `gemini-flash-latest`) for two features: automatic Bengali product-description generation on the seller dashboard, and the customer-facing AI shopping assistant chatbot, both implemented in `src/services/geminiService.ts`.

jsPDF and html2canvas

Invoices and delivery slips are generated entirely on the client: a hidden, styled HTML element representing the document is rendered off-screen, captured as a canvas image using `html2canvas`, and inserted into a PDF document using `jsPDF`, which is then offered to the user as a download (`src/lib/pdfGenerator.ts`).

5.4 Project Structure

The relevant parts of the project's folder structure are summarized below (build artefacts, dependency folders, and version-control metadata omitted):

```
karumart/
├── server.ts          # Express server / Vite middleware wrapper
├── database_setup.sql # Baseline Supabase/PostgreSQL schema and RLS policies
├── src/
│   ├── App.tsx        # Route definitions
│   ├── main.tsx       # React application entry point
│   ├── contexts/
│   │   ├── AuthContext.tsx # Session, profile loading, sign-out
│   │   ├── CartContext.tsx  # Cart items, totals, vouchers, delivery fee
│   │   └── LanguageContext.tsx # Bengali/English language state
```

```

|   |— services/
|   |   |— paymentService.ts # Simulated bKash / Nagad / SSLCommerz / COD
|   |   |— geminiService.ts  # AI description generator + chat assistant
|   |   |— adminService.ts   # Admin action audit logging
|   |— lib/
|   |   |— supabase.ts       # Supabase client
|   |   |— searchUtils.ts    # Bilingual/Banglish search normalization
|   |   |— translations.ts   # Bengali/English translation dictionary
|   |   |— districts.ts      # Bangladesh division/district reference data
|   |   |— pdfGenerator.ts   # Client-side PDF export (invoices, slips)
|   |— components/          # Reusable UI (Layout, ProductCard, modals, ui/*)
|   |— pages/
|   |   |— Home.tsx, ProductDetail.tsx, ShopDetail.tsx, Cart.tsx, Checkout.tsx
|   |   |— Login.tsx, Register.tsx, AdminLogin.tsx
|   |   |— Dashboard/
|   |       |— ConsumerDashboard.tsx
|   |       |— SellerDashboard.tsx
|   |       |— DeliveryDashboard.tsx
|   |       |— AdminDashboard.tsx
|   |— types/index.ts       # Shared TypeScript domain types
|— package.json

```

5.5 Frontend Implementation

Client-side routing is defined once in `App.tsx` using React Router's declarative `<Routes>/<Route>` structure, wrapped in `LanguageProvider`, `AuthProvider`, `CartProvider`, and a shared `Layout` component that renders the persistent navigation bar, search box, and footer around every page. Protected routes are wrapped in the `ProtectedRoute` component described in Section 4.9. Each dashboard role (consumer, seller, delivery man, admin) is implemented as a separate top-level page component, selected at runtime by `Dashboard/index.tsx` based on the logged-in user's `profile.role`, so that a single `/dashboard` route serves four structurally different experiences without exposing role-specific URLs.

Forms throughout the application are built as controlled components using React's `useState` hook, with validation performed inline (for example, checking that a product's quantity does not exceed available stock before allowing an Add to Cart action) and user feedback delivered through toast notifications (the `sonner` library) rather than blocking browser alerts.

5.6 Backend Implementation

Karumart does not implement a custom backend API layer of its own routes and controllers in the traditional sense. Business logic that would, in a conventional three-tier web application, live in server-

side controllers is instead implemented directly in the frontend's context providers and service modules, which call Supabase's auto-generated PostgREST API and enforce data-access rules through PostgreSQL Row Level Security policies rather than through custom server middleware. The only server-side process in the project, `server.ts`, is a thin Express wrapper whose sole responsibilities are to run Vite's development middleware and to serve the compiled static build in production; it does not define any `/api` routes. This is an accurate characterization of the current implementation and should be understood as a deliberate architectural choice enabled by Supabase, rather than a missing component.

5.7 Database Implementation

The database schema is provisioned by running `database_setup.sql` directly in the Supabase SQL editor, which creates the ten baseline tables described in Section 4.6, enables Row Level Security on each, and defines an initial set of public-read and authenticated-insert policies. Foreign keys (for example, `products.farmer_id` referencing `profiles.id`, and `order_items.order_id` referencing `orders.id` with cascading delete) enforce referential integrity at the database level. All application queries use Supabase's JavaScript client, which builds parameterized PostgREST requests, reducing the risk of SQL injection compared to hand-built query strings.

5.8 Authentication and Authorization

Authentication is implemented in `AuthContext.tsx`, which, on load, asks Supabase for the current session, and if one exists, fetches the corresponding row from the `profiles` table to determine the user's role and other profile fields. The context also subscribes to Supabase's `onAuthStateChange` event so that login and logout occurring anywhere in the application are reflected everywhere immediately. If the Supabase environment variables are missing or misconfigured, the provider renders a dedicated Bengali-language configuration-error screen rather than failing silently.

```
const { data, error } = await supabase
  .from('profiles')
  .select('*')
  .eq('id', userId)
  .single();
if (error) throw error;

setProfile(data); // role, shop_status, is_frozen, badges, etc.
```

Authorization then operates at two levels: the `ProtectedRoute` component checks the logged-in `profile.role` against an `allowedRoles` list before rendering a route, and, independently, PostgreSQL Row Level Security policies restrict which rows a given authenticated user may actually read or write at the database level — so even if a client-side check were bypassed, the database itself limits access.

5.9 Major Module Implementation

5.9.1 Home Page

`Home.tsx` renders the hero banner carousel, a trust-badge strip, a promotional banner, a flash-sale section with a live countdown timer, category browsing, and the main paginated product grid. It supports keyword search, category filtering, and district-based filtering of sellers, using helper functions from `searchUtils.ts` and `districts.ts`.

5.9.2 User Registration and Login

Register.tsx lets a new user choose between Consumer, Seller, and Delivery Man roles, then calls `supabase.auth.signUp()` followed by an insert into the profiles table; seller accounts are created with status: 'pending' so that they require admin approval before their dashboard becomes fully active, while consumer and delivery-man accounts default to an approved status. Login.tsx and AdminLogin.tsx authenticate against the same Supabase Auth backend, with the admin login page additionally verifying that the authenticated profile's role is 'admin' before granting access to the admin dashboard route.

5.9.3 Product Browsing and Detail

ProductDetail.tsx displays full product information, an image gallery, stock-aware Add to Cart / Buy Now controls, a wishlist toggle, a share action, related products from the same category, and the product's existing reviews. ShopDetail.tsx renders a seller's public storefront: shop banner, trust badges (Best Seller, Verified/Official, Open/Closed status), and the seller's full product listing.

5.9.4 Search and Filtering

searchUtils.ts implements a normalization function that maps common English and Banglish (Romanized Bengali) terms — for example "saree", "sari", and "shari" — to their canonical Bengali equivalents, so that a search for any of these variants matches the same underlying products. Combined with category and district filters on the Home page, this gives consumers a search experience that does not require typing in Bengali script to find Bengali-named products.

5.9.5 Cart

CartContext.tsx holds cart items in React state, mirrored to the browser's localStorage so that a cart survives a page reload. It computes subtotal, delivery fee (waived above a configurable free-delivery threshold fetched from site_settings, with real-time updates via a Supabase channel subscription), any applied voucher discount, and the resulting grand total, exposing these as ready-to-render values to the Cart and Checkout pages.

```
const subtotal = items.reduce((sum, item) => {
  const activePrice = (item.flash_sale_active && item.flash_sale_price)
    ? item.flash_sale_price : item.price;
  return sum + (activePrice * item.cartQuantity);
}, 0);
const deliveryFee = subtotal >= settings.free_delivery_threshold || items.length === 0
  ? 0 : settings.delivery_charge;
```

5.9.6 Checkout and Payment

Checkout.tsx re-validates stock and shop status per item, creates the order and order_items records, invokes PaymentService for non-COD payment methods, and decrements stock. PaymentService.ts implements a unified processPayment(method, amount, orderId) function that dispatches to a simulated bKash, Nagad, or SSLCommerz flow — each returning a mock transaction ID after an artificial delay — or, for Cash on Delivery, returns success immediately without requiring a transaction ID. The module's own inline documentation explicitly describes these as simulations of real-world Bangladeshi payment gateways rather than live integrations.

```

async processPayment(method, amount, orderId) {
  switch (method) {
    case 'bkash': return this.processBkashPayment(amount, orderId);
    case 'nagad': return this.processNagadPayment(amount, orderId);
    case 'card': return this.processSSLCommerzPayment(amount, orderId);
    case 'cod': return { success: true };
    default: return { success: false, error: 'Invalid payment method' };
  }
}

```

5.9.7 Consumer Dashboard

ConsumerDashboard.tsx is organized into five tabs — Overview, Orders, Wishlist, Address, and Settings. The Orders tab lists past purchases with a status stepper, and offers actions to download an invoice (InvoiceModal), submit a complaint or return (ComplaintModal), or rate a delivered order (RatingModal).

5.9.8 Seller Dashboard

SellerDashboard.tsx is the largest consumer-facing dashboard after the admin panel. It supports adding a product (with an optional one-click Gemini-generated Bengali description), editing stock, price, category, description, and SKU, deleting products, viewing and fulfilling incoming orders, and managing shop profile fields including logo, cover image, address (with district auto-detection), and shop status (open, temporarily closed, permanently closed).

5.9.9 Delivery Dashboard

DeliveryDashboard.tsx presents a delivery person with a filterable list of available and already-assigned orders, filterable by pickup district and assigned district, and provides one-tap status transitions (Picked Up → On the Way → Delivered, with cancellation and return handling) along with printable delivery slips generated through DeliverySlipModal.

5.9.10 Admin Dashboard

AdminDashboard.tsx is organized into thirteen tabs: Overview, Sellers, Orders, Complaints, Vouchers, Flash Sale, Products, Admin Logs, Slider Banners, Side Promo Banners, Bottom Static Banners, New Admin, and Settings. The Overview tab renders sales trend charts, an order-status breakdown, a top-five-products chart, and a category sales breakdown using Recharts, sourced from live aggregation of the orders and products tables.

5.9.11 AI Chatbot and Description Generator

AiChatbot.tsx renders a floating chat widget available across the site. On open, it fetches the current site contact settings and a snapshot of approved, in-stock products (name, price, quantity, seller) to give the Gemini model grounded context, then streams the visitor's question and the assistant's response through chatWithSupport() in geminiService.ts, with a system instruction that keeps the assistant scoped to Karumart-related queries and defers to the platform's hotline or email when it cannot answer.

generateProductDescription() is called from the seller dashboard's Add/Edit Product form to produce a short Bengali description from a product's name and category alone.

5.10 Important Code Explanation

The Checkout stock-and-shop validation step below is one of the more important pieces of business logic in the codebase, since it is the last line of defence preventing an order from being placed against a product that has since sold out or whose seller has closed or been frozen:

```
const { data: product } = await supabase
  .from('products')
  .select('quantity, name, farmer:profiles!inner(shop_status, is_frozen)')
  .eq('id', item.id).single();
const farmer = product.farmer;
if (farmer?.shop_status !== 'open' || farmer?.is_frozen) {
  throw new Error(farmer?.is_frozen ? 'শপটি ফ্রিজ করা হয়েছে' : t.shopClosedError);
}
if (product.quantity < item.cartQuantity) {
  throw new Error(`${product.name} ${t.insufficientStock}${product.quantity}`);
}
```

This query uses a Supabase relational select (profiles!inner(...)) to join each product to its seller's profile in a single round trip, so that stock and shop status can be checked together for every cart item before any order record is written — an example of the kind of business-rule enforcement that, in a traditional server-rendered application, would sit in a backend controller, but here sits directly in the frontend's checkout handler, backed by database-level Row Level Security as a second line of defence.

5.11 Chapter Summary

This chapter documented the technology stack, project structure, and implementation of every major Karumart module, from authentication and the shopping cart through to the seller, delivery, and admin dashboards and the Gemini-powered AI features. A defining architectural characteristic of the implementation is the absence of a custom backend API layer: business logic is implemented in the frontend and enforced at the database layer through Supabase and PostgreSQL Row Level Security, rather than through server-side controllers. The next chapter presents the system's interface through the project's design wireframes.

Chapter 6: System Interface (Live Application Screenshots)

6.1 Introduction

This chapter presents the Karumart interface through screenshots captured directly from the live, deployed application at karumartbdttest.netlify.app. Sections 6.2–6.8 follow a consumer account ("kamrul islam") through a typical shopping journey: home page, product search, cart, wishlist, dashboard, order tracking, and product recommendations. Section 6.9 then presents thirteen further screenshots captured from the Main Admin account, covering seller management, order oversight, vouchers, flash sales, product moderation, the audit log, banner management, admin accounts, site settings, and platform analytics. All twenty screenshots are genuine captures of the running system rather than design mockups. The project's original design wireframes, used to plan these screens before implementation, are presented afterward in Section 6.10 for comparison.

6.2 Home Page

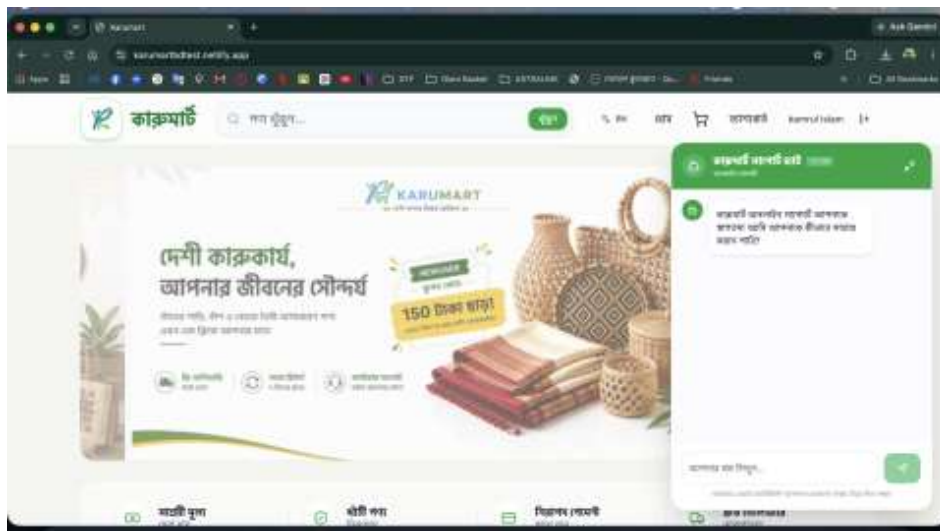


Figure 6.1: Karumart Home Page — Live Application

The live home page matches the structure planned in the design wireframe: a top navigation bar (logo, search bar, language switch, cart, dashboard, and account menu), a hero banner promoting handloom sarees and bamboo/cane products with a new-user discount coupon, and trust-signal icons (affordable pricing, authentic products, secure payment, fast delivery) below the fold. The floating AI support chat widget ("কারুমাট সাপোর্ট চ্যাট") is visible in the bottom-right corner, opened here to show its Bengali-language welcome message, confirming the Gemini-powered chatbot described in Section 5.9.11 is live and functioning.

6.3 Product Search

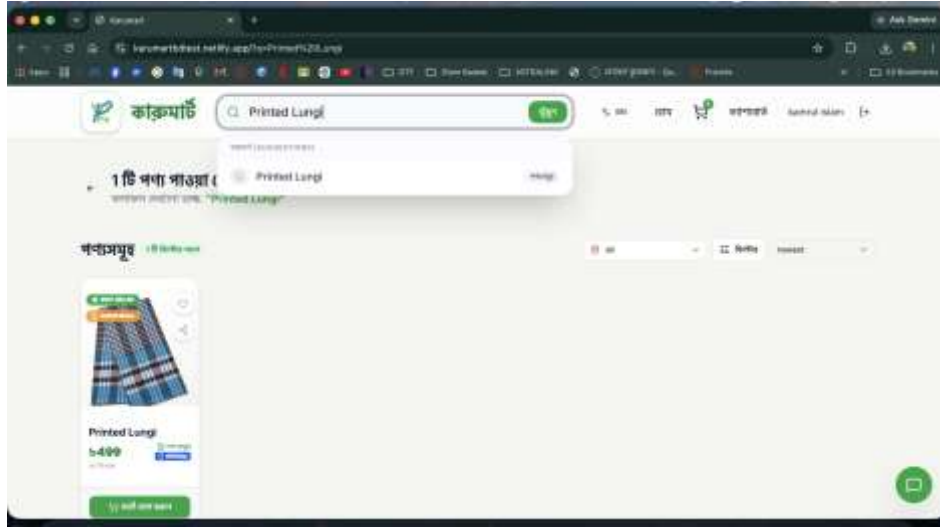


Figure 6.2: Search Results for "Printed Lungi"

Searching the English keyword "Printed Lungi" against the live catalog returns the matching product card, confirming that the bilingual/Banglish search logic described in Section 5.9.4 correctly matches an English-language query to a Bengali-catalogued product. The result card displays the Best Seller and Super Mall badges, the current price (৳499), stock count (197 টি আছে), an OFFICIAL seller verification badge, and an Add to Cart action — all consistent with the product card design described in Section 4.8.

6.4 Shopping Cart

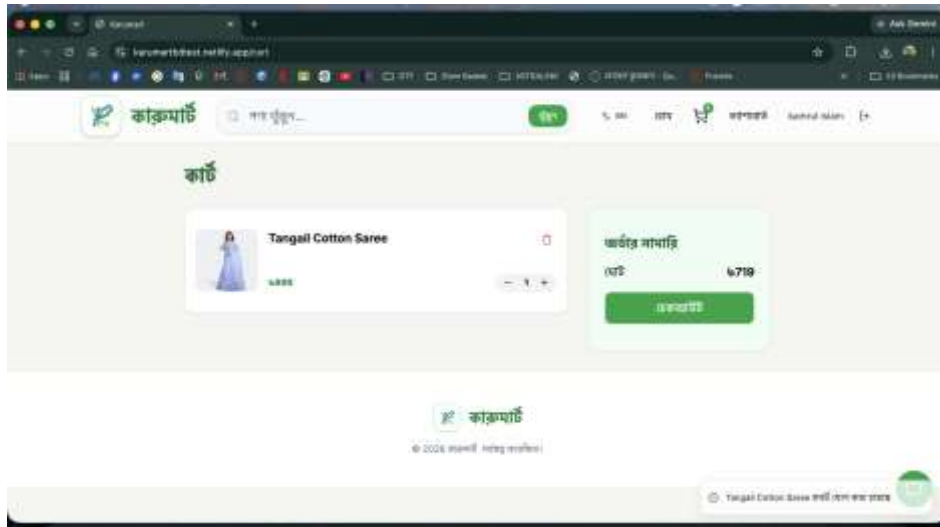


Figure 6.3: Shopping Cart with Applied Item

The cart page shows a Tangail Cotton Saree added at a quantity of 1, with a quantity stepper, a remove (trash) action, and an order summary panel showing the total (৳719, reflecting the item's active discounted price) and a Checkout (চেকআউট) button. A toast notification in the bottom-right corner ("Tangail Cotton Saree কার্টে যোগ করা হয়েছে") confirms the add-to-cart action succeeded — direct evidence that the CartContext logic described in Section 5.9.5 is functioning correctly end-to-end in the deployed build.

6.5 Wishlist

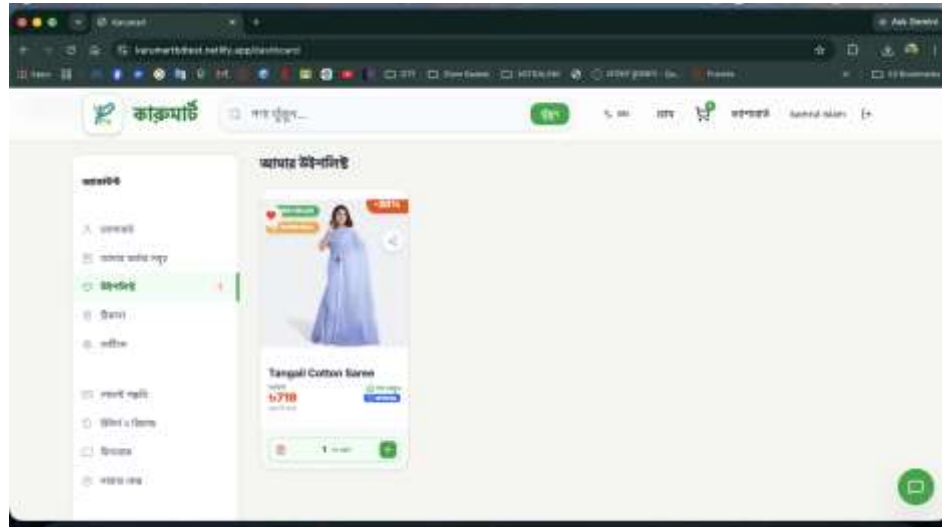


Figure 6.4: Consumer Wishlist

The wishlist tab of the consumer dashboard lists the same Tangail Cotton Saree, showing its original price (৳899, struck through), discounted price (৳719), a 20% discount badge, Best Seller and Super Mall badges, and an indicator that the item is already in the cart ("1 in cart") with an inline quantity control — confirming that the wishlist and cart states are correctly synchronized for a given product.

6.6 Consumer Dashboard Overview

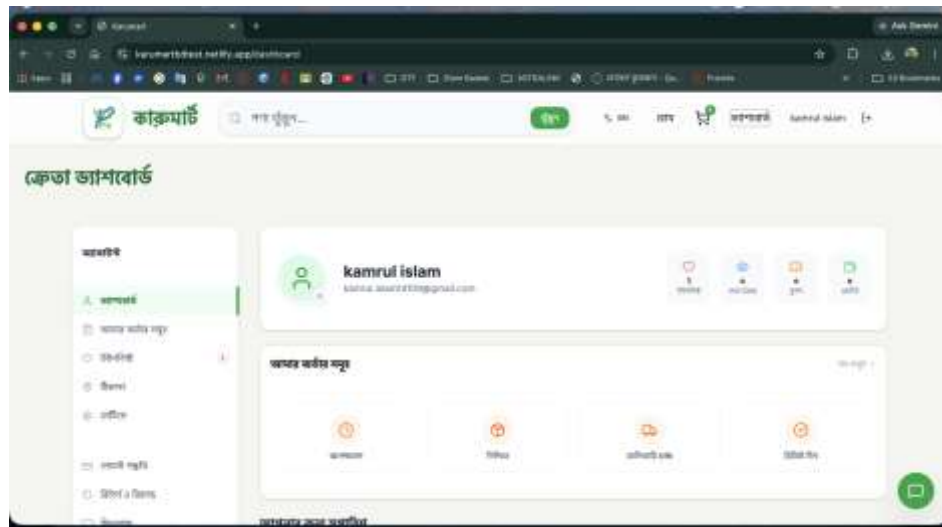


Figure 6.5: Consumer Dashboard — Overview Tab

The dashboard overview tab (ক্রেতা ড্যাশবোর্ড) greets the logged-in consumer by name and email, and shows four summary counters — wishlist count, viewed items, coupons, and credit — followed by an order-status summary row (Pending, Confirmed, Delivering, Review). This matches the tabbed ConsumerDashboard.tsx structure (Overview, Orders, Wishlist, Address, Settings) described in Section 5.9.7, and confirms the account correctly shows one wishlist item, consistent with Figure 6.4.

6.7 Order Tracking

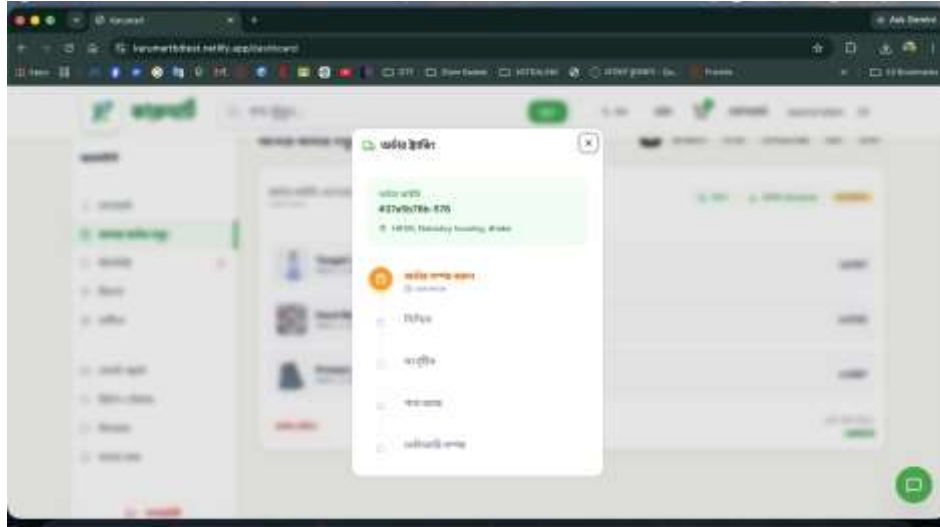


Figure 6.6: Order Tracking Modal

Selecting Tracking on a placed order opens a status modal showing the order ID (#37a5b78b-578), the delivery address (H#1/B, Nobodoy housing, dhaka), and a five-step vertical status tracker — Order Placed (অর্ডার সম্পন্ন করুন, dated ১৬/৮/২০২৬ and highlighted as the current step), Confirmed, Picked Up, On the Way, and Delivery Complete. This is a direct, working demonstration of the order-status lifecycle described in Section 4.6 and Section 5.9.6, captured from an order actually placed on the live system rather than inferred from source code alone.

6.8 Product Recommendations

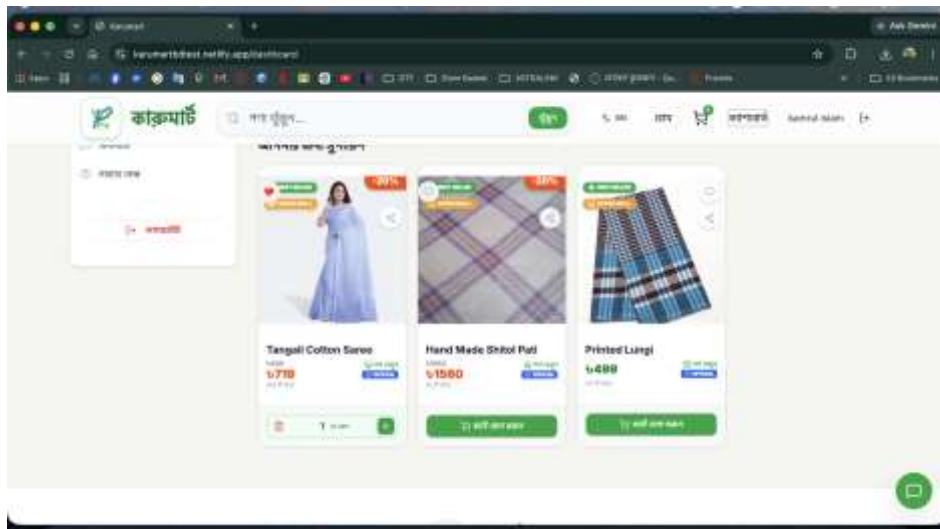


Figure 6.7: "Recommended for You" Product Grid

The bottom of the dashboard surfaces a recommended-products grid (আপনার জন্য সুপারিশ) showing three products — Tangail Cotton Saree, Hand Made Shitol Pati, and Printed Lungi — each with current and struck-through original prices, stock counts, Best Seller/Super Mall/Official badges, and Add to Cart buttons. The first card correctly reflects that this item is already in the cart, showing the same quantity stepper seen in Figure 6.3 rather than a plain Add to Cart button, which confirms that cart state is read consistently across every part of the interface that renders a product card.

6.9 Admin Dashboard

Beyond the consumer-facing journey documented above, the administrator control panel was also captured directly from the live application, logged in as the platform's Main Admin account. This section presents thirteen screenshots covering the full range of admin functionality described in Section 5.9.10, confirming that the seller-approval pipeline, order oversight, promotions, content management, and platform-configuration tools are implemented and operating on the deployed system rather than existing only as design intent.

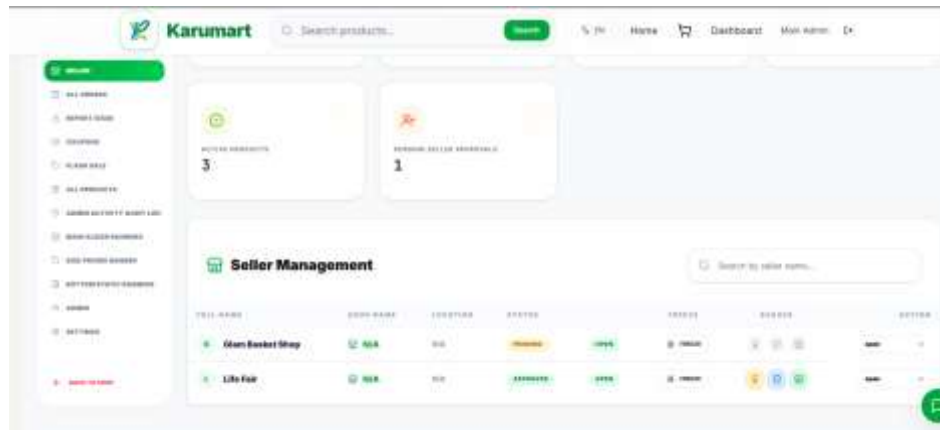


Figure 6.8: Admin Overview and Seller Management

The admin Seller tab lists every registered shop with its approval status, open/closed state, freeze control, badge toggles, and a per-row action menu. In this capture, "Glam Basket Shop" is shown with a Pending status awaiting approval, while "Life Fair" is already Approved and carries Best Seller, Official, and Karu Mall badges — directly demonstrating the seller-approval workflow (FR-27) and badge-assignment feature (FR-32) described in Chapter 3. The summary cards above the table (Active Products: 3, Pending Seller Approvals: 1) confirm the platform's live moderation queue.

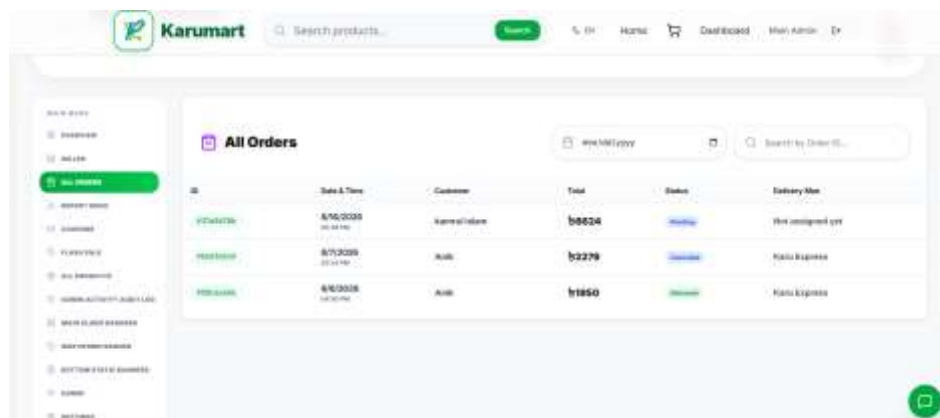


Figure 6.9: Admin All Orders View

The All Orders tab lists every order platform-wide with its ID, date, customer, total, status, and assigned delivery man, filterable by date or Order ID. The order #37a5b78b visible here is the same order shown from the consumer's perspective in Figure 6.6, confirming that order data is consistent between the consumer and admin views of the system.

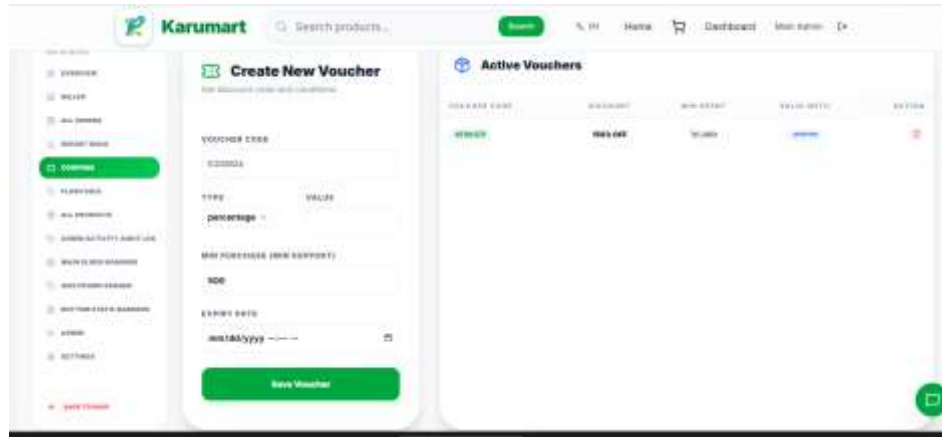


Figure 6.10: Voucher / Coupon Management

The Coupons tab lets an admin create a new voucher (code, discount type and value, minimum purchase, and expiry date) and lists currently active vouchers. The active "NEWUSER" voucher shown here (₹1500 off, minimum spend ₹1,000) is the same promotional code advertised on the home page hero banner in Figure 6.1, demonstrating that the voucher system documented in FR-17 and FR-30 operates end-to-end from creation to consumer-facing promotion.

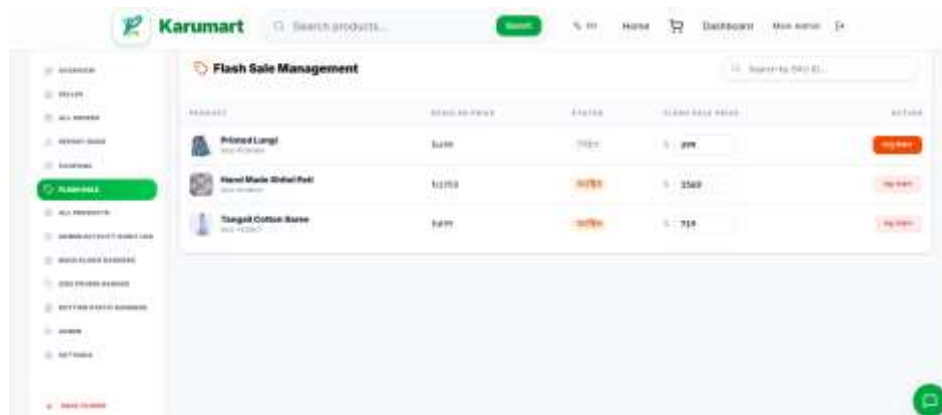


Figure 6.11: Flash Sale Management

The Flash Sale tab lists every product with its regular price, an editable flash-sale price field, an active/inactive status toggle (shown here in Bengali as **আ্যক্টিভ/নিষ্ক্রিয়**), and an activation action button. This is the administrative counterpart to the consumer-facing flash-sale countdown described in Section 4.3 and FR-14.

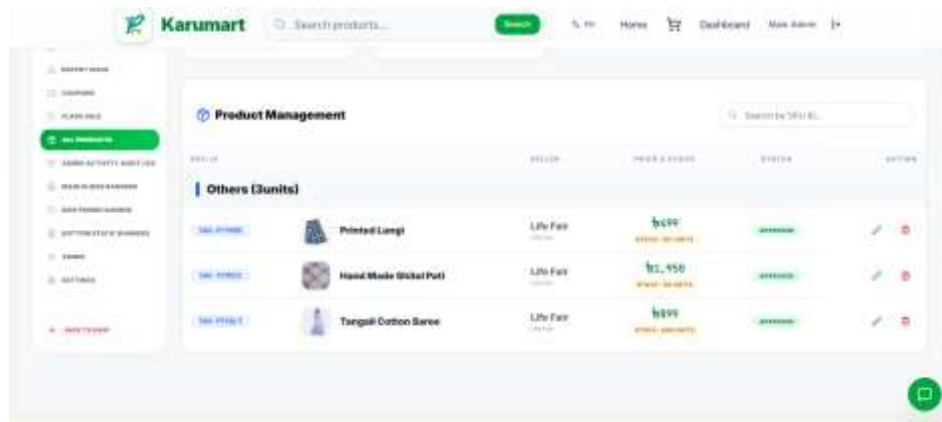


Figure 6.12: Product Management (All Products)

The All Products tab lists the full catalog grouped by category, showing SKU, seller, price, stock, and approval status, with edit and delete actions per row. The three products visible here (Printed Lungi, Hand Made Shitol Pati, Tangail Cotton Saree, all under seller "Life Fair") are the same catalog items that appear throughout the consumer-facing screenshots in this chapter, confirming a single consistent product catalog across both admin and consumer views.

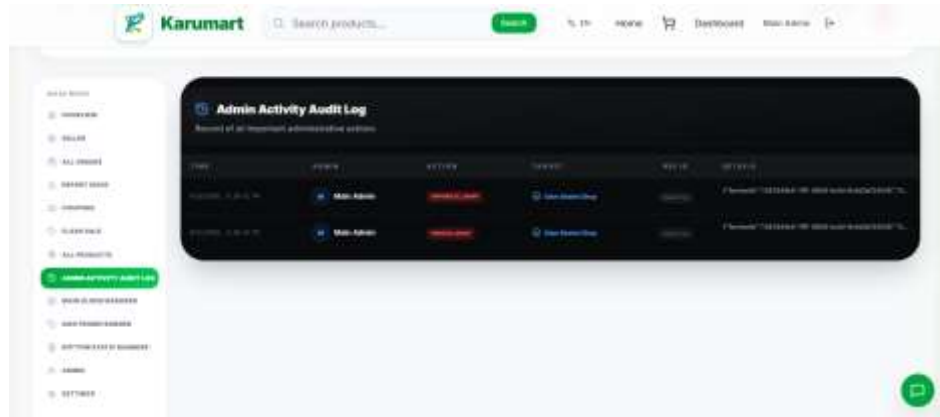


Figure 6.13: Admin Activity Audit Log

The Admin Activity Audit Log records administrative actions with a timestamp, the acting admin, the action taken, the target entity, a reference ID, and a JSON details payload. The two entries shown here — an UNFREEZE_SHOP action immediately following a FREEZE_SHOP action, both against "Glam Basket Shop" — provide direct, first-hand evidence that the shop-freeze mechanism described in FR-29 and Section 4.9 is implemented and that every such action is captured in the audit trail required by FR-35.

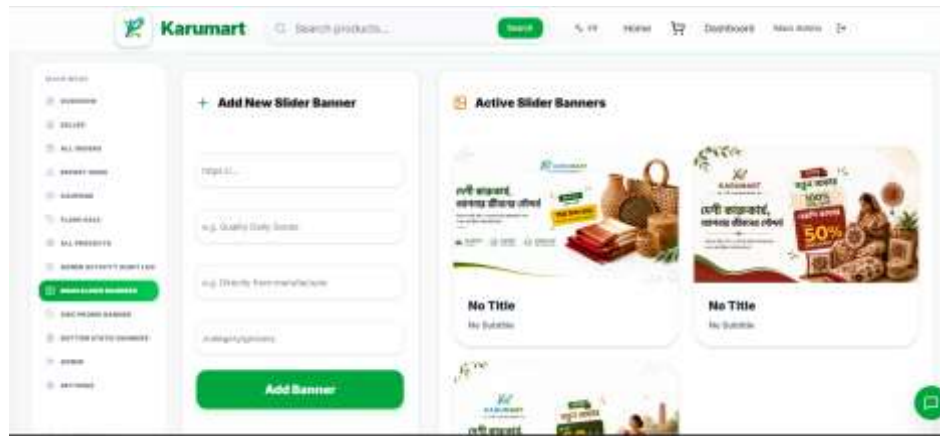


Figure 6.14: Main Slider Banner Management

The Main Slider Banners tab lets an admin add a new homepage carousel banner (image URL, title, subtitle, and link) and shows currently active banners. The two active banners visible here are the same promotional slides that appear in the live hero carousel captured in Figure 6.1.

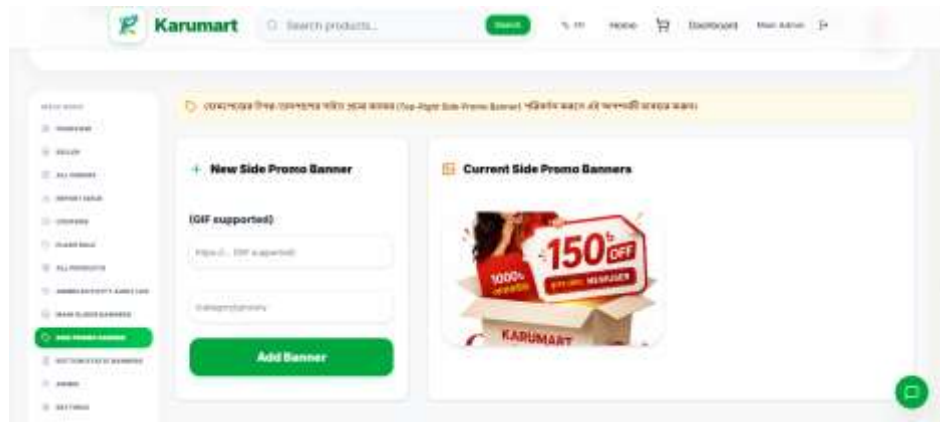


Figure 6.15: Side Promotional Banner Management

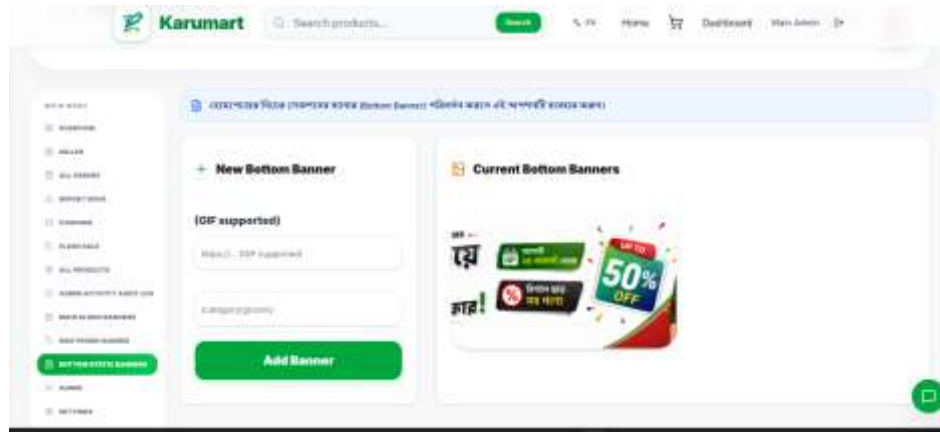


Figure 6.16: Bottom Static Banner Management

The Side Promo Banner and Bottom Static Banner tabs provide equivalent add/preview controls for the two remaining homepage promotional slots described in FR-31, each with Bengali-language help text explaining which part of the homepage the banner controls.

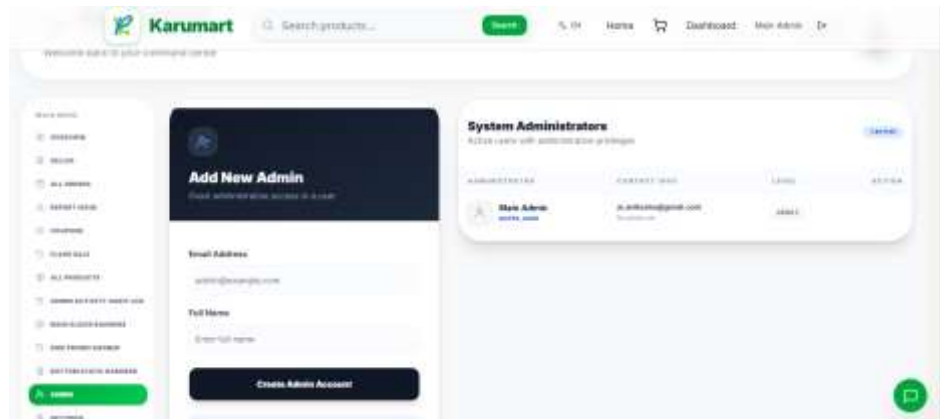


Figure 6.17: Administrator Account Management

The Admin tab lets an existing administrator create additional admin accounts (email and full name) and lists current System Administrators with their contact information and access level. At the time of capture, exactly one administrator ("Main Admin", SUPER_USER level) is active, consistent with FR-36.

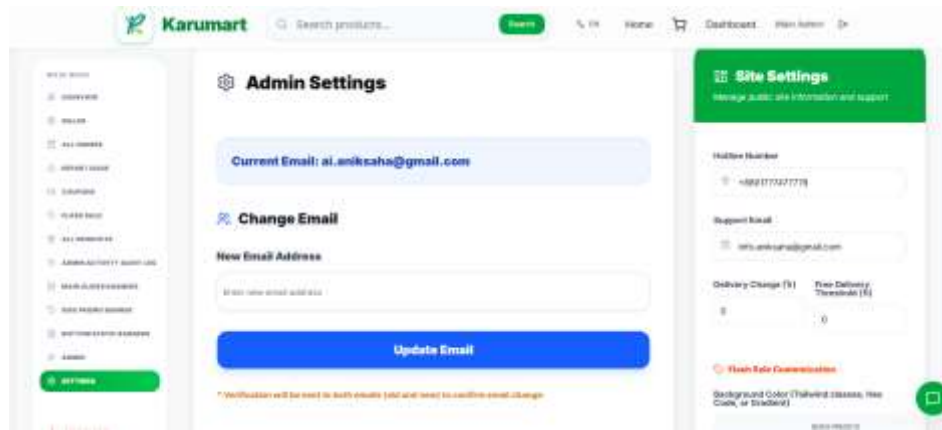


Figure 6.18: Admin and Site Settings

The Settings tab is split into account settings (change the admin's own login email, with verification sent to both the old and new address) and platform-wide Site Settings (support hotline, support email, delivery charge, and free-delivery threshold), plus flash-sale display customization. This is the live counterpart to the `site_settings` database table described in Section 4.6 and directly implements FR-37; the delivery-charge and free-delivery-threshold fields shown here are the same values consumed by the `CartContext` calculation described in Section 5.9.5.



Figure 6.19: Admin Overview — Analytics Charts

The Overview tab's analytics section renders a Monthly Orders & Revenue bar chart, an Order Status donut chart, a Sales by Category donut chart, a Daily Sales Trend panel, and a Top 5 Best Selling Products list (topped here by "Hand Made Shitol Pati" at 3,510 across 2 units) — implementing the platform-statistics requirement FR-33 using the `Recharts` library described in Section 5.9.10.

ID	Date & Time	Customer	Total	Status	Delivery Note
1	8/8/2020 12:00:00	Indrani Kaur	12000	Pending	Not assigned yet
2	8/7/2020 12:00:00	Anu	10000	Pending	Not assigned yet
3	8/6/2020 12:00:00	Anu	10000	Pending	Not assigned yet

Figure 6.20: Admin Panel (Alternate / V2.0 Build) — All Orders

This final capture shows the same All Orders data (order #37a5b78b and two earlier orders) rendered in what is labelled "Admin Panel V2.0 PRO — System Monitoring" with a slightly different header and sidebar treatment than Figure 6.9. [REVIEW NEEDED: this screenshot shows a visibly different admin panel header/branding ("Admin Panel V2.0 PRO", a "Main Admin / SUPER ADMIN" badge, and an avatar) than the other twelve admin screenshots in this section. It most likely reflects either a newer build captured at a different point in development or a distinct admin theme/version, but this could not be confirmed from the available project files. The underlying order data is identical to Figure 6.9, so no functional discrepancy is implied — only a cosmetic/versioning one worth confirming with the development team before final submission.]

6.10 Design Wireframes (Reference)

The following wireframes, from the project's original "Overall Architecture" design document, were used to plan the screens above — including the admin dashboard, now confirmed by the live screenshots in Section 6.9, and the seller dashboard described in Section 5.9.8 — before implementation. They are included here for reference and to show the correspondence between the original design intent and the delivered interface.

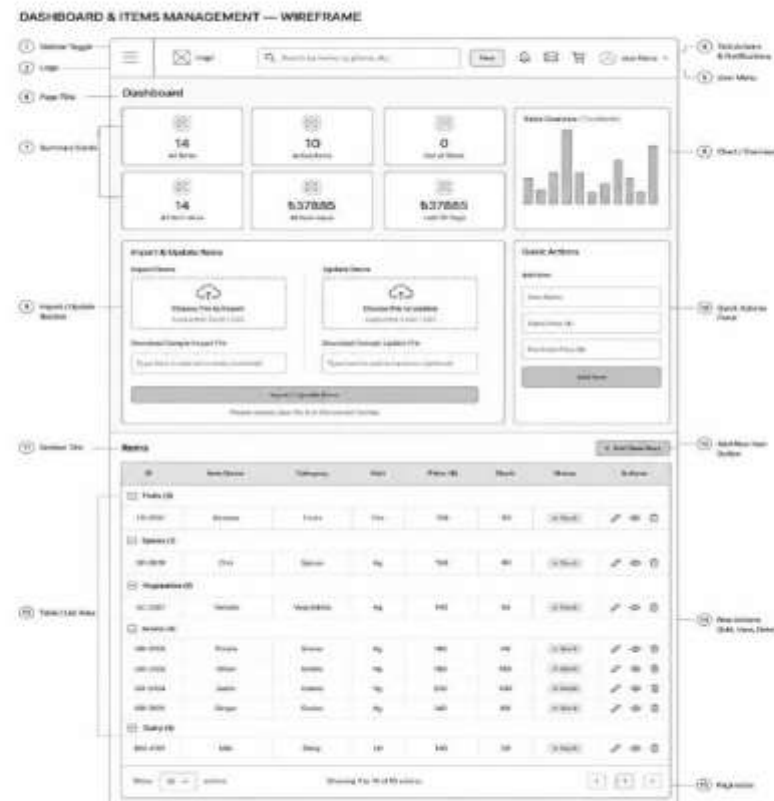


Figure 6.21: Seller Dashboard and Item Management Wireframe



Figure 6.22: Admin Dashboard Wireframe



Figure 6.23: Shop / Seller Storefront Wireframe

6.11 Responsive Design – Laptop and Mobile Friendly Views

Karumart is implemented as a responsive web application, with the interface adapting to desktop/laptop and mobile screen sizes. The following two views demonstrate the same Karumart home-page experience optimized for different device form factors, supporting the responsiveness requirement described in Section 3.4.

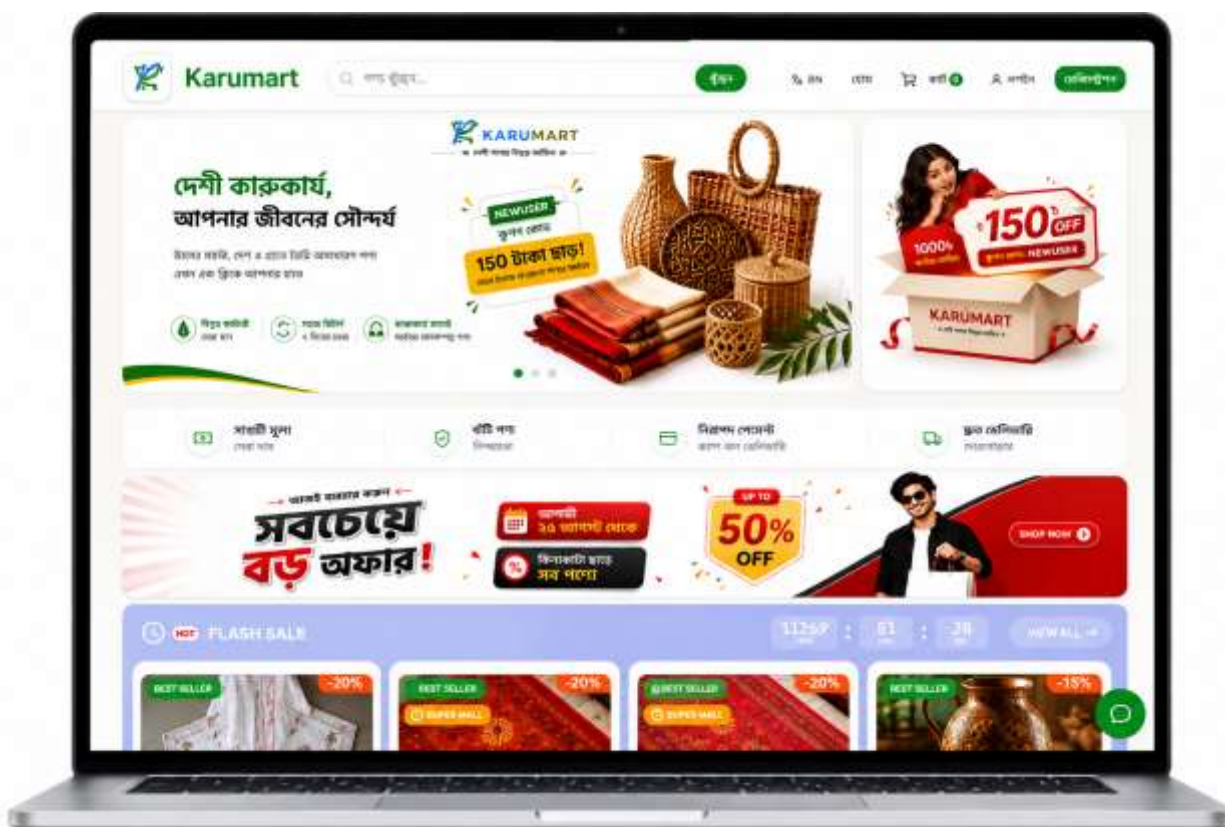


Figure 6.24: Karumart Laptop-Friendly Home Page



Figure 6.25: Karumart Mobile-Friendly Home Page

6.12 Chapter Summary

This chapter presented twenty screenshots captured directly from the live Karumart application: seven tracing a consumer's journey from the home page through search, cart, wishlist, dashboard, order tracking, and product recommendations, and thirteen more covering the full administrator control panel — seller management, order oversight, vouchers, flash sales, product moderation, the audit log, banner management, admin accounts, site settings, and platform analytics — followed by the project's original seller, admin, and storefront wireframes and the laptop- and mobile-friendly responsive views for reference. This chapter demonstrates that the core consumer-facing and administrative functionality described in Chapters 4 and 5 is implemented and operating correctly on a deployed build of the system, not merely planned in design documentation.

Chapter 7: Testing and Evaluation

7.1 Introduction

Testing verifies that a system behaves as intended and helps surface defects before they reach real users — a particular concern for Karumart, where checkout, stock, and payment logic directly affect real transactions. This chapter summarizes the testing performed and planned for the project, drawing on two internal testing documents supplied with the project (a unit/context-level testing report and an integration testing report) together with a set of functional test cases derived directly from the application's implemented features.

7.2 Testing Strategy

The project's own documentation describes two complementary layers of testing. The first is an automated unit and context-level test suite, run using the Vitest framework and covering shared utilities, Bangladesh district data, multilingual search, translations, TypeScript type definitions, the payment service, and the Auth/Card/Language context providers. The project's internal testing report describes this suite as comprising ten test files and approximately 201 test cases, all passing. A screenshot of an actual terminal test run, captured directly from the development environment and reproduced in Figure 7.1, provides independent, first-hand evidence that this suite exists and executes — but the observed run reports different totals from the internal report's summary, a discrepancy discussed in Section 7.3.

The second layer is a formally scoped integration test plan, covering authentication, cart and checkout, payment-method routing, search, multilingual switching, admin operations, dashboard access, and voucher/discount calculation. This plan explicitly records twenty integration test cases (IT-01 through IT-20) with their expected results, but its own status column is marked "To Be Verified" throughout — the plan itself states that actual pass/fail results must come from an executed test run, and no such execution record was supplied. This report preserves that status accurately rather than presenting the integration plan as completed testing.

7.3 Automated Unit and Context-Level Testing — Actual Execution Evidence

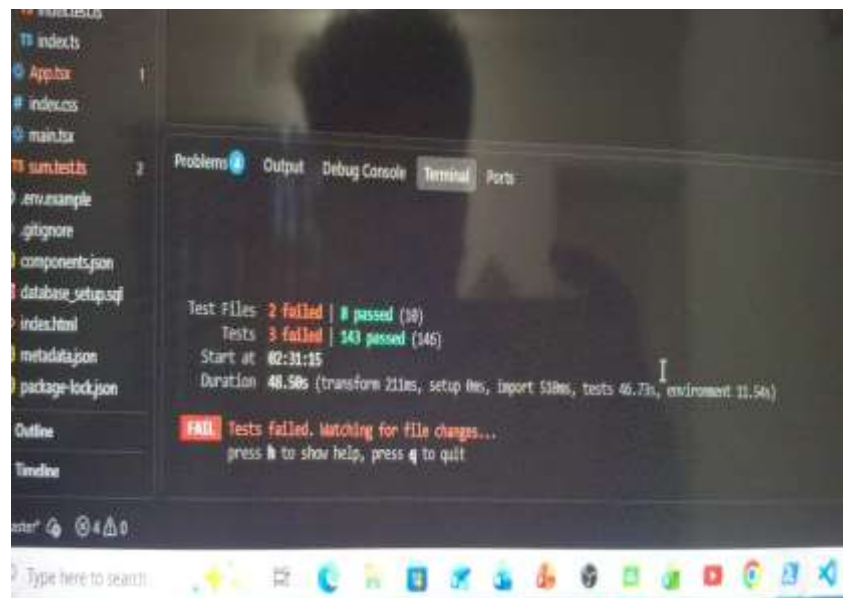


Figure 7.1: Actual Vitest Terminal Output from the Karumart Development Environment

Figure 7.1 shows an actual terminal run of the project's automated test suite in VS Code (visible file tree includes `index.test.ts`, `sum.test.ts`, `App.tsx`, `database_setup.sql`, and other real project files, confirming this is the Karumart repository rather than a placeholder project). The observed output reads:

- Test Files: 2 failed | 8 passed (10 total)
- Tests: 3 failed | 143 passed (146 total)
- Run started at 02:31:15; total duration 48.50 seconds (transform 211ms, setup 0ms, import 510ms, tests 46.73s, environment 11.54s)
- Final status: "FAIL — Tests failed. Watching for file changes..."

This is a materially more complete and more honest picture than could be given from documentation alone. Two points are worth noting explicitly. First, this screenshot confirms that the ten test files described in the project's internal testing report do exist and do execute in a real Vitest environment — the earlier concern that these files were absent from the reviewed source tree is resolved by this evidence; they were simply not included in the specific project archive originally supplied for this report. Second, the observed totals (10 files, 146 tests, 3 failing) differ from the internal testing report's own summary (10 files, approximately 201 tests, all passing). The most likely explanation is that Figure 7.1 was captured at an earlier or later point in development than the run the internal report summarizes, and/or that the suite has grown or changed since; regardless of the cause, this report treats the directly observed screenshot in Figure 7.1 as the more reliable source, since it is primary evidence of an actual execution rather than a narrative description.

Table 7.0: Unit / Context-Level Test Suite — Reported Composition vs Directly Observed Run

Source	Test Files	Total Tests	Passing	Failing
Internal testing report (narrative description)	10	~201	201 (100%)	0 (documented as none)
Figure 7.1 — actual terminal screenshot	10 (2 failed, 8 passed)	146	143 (98.0%)	3 (2.0%)

Taken together, the two sources agree on the shape of the suite (ten files spanning utilities, types, services, and state-management contexts, as listed in the internal report and summarized in Section 7.3.1 below) but disagree on the precise pass count. A 98% pass rate across 146 executed tests, with 3 identified failures still open in 2 files, is a strong — if not perfect — result, and is reported here as such rather than being rounded up to "all passing."

7.3.1 Test Suite Composition (as described in project documentation)

Table 7.0b: Unit / Context-Level Test File Composition

Test File / Module	Approx. Test Count (documented)
<code>utils.test.ts</code> (Tailwind class-name merge helper)	8
<code>districts.test.ts</code> (Bangladesh division/district data)	10

Test File / Module	Approx. Test Count (documented)
searchUtils.test.ts (multilingual/Banglish search)	25+
translations.test.ts (Bengali/English label coverage)	20+
index.test.ts (TypeScript type/role/status definitions)	40+
paymentService.test.ts (bKash / Nagad / SSLCommerz / COD)	15+
AuthContext.test.ts	17
CartContext.test.ts	30+
LanguageContext.test.ts	35+
sum.test.ts (baseline arithmetic check)	1

This suite validates shared utilities, data structures, type definitions, and state-management logic rather than rendered UI behaviour or browser interactions — those areas remain covered only by the manual functional test cases in Section 7.4 and the not-yet-executed integration plan referenced in Section 7.2. Identifying and resolving the 2 failing files / 3 failing tests visible in Figure 7.1 is a concrete, actionable next step, and is carried forward as a recommendation in Section 8.5.

7.4 Functional Testing

The table below lists functional test cases derived directly from Karumart's implemented features. Because a live, executed QA pass was not documented for this specific test set, results are marked "To be verified" rather than assumed to pass, consistent with the accuracy standard applied throughout this report.

Table 7.1: Functional Test Cases

Test Case ID	Module	Test Scenario	Expected Result	Status
FT-01	Auth	Register a new consumer account	Account is created and status defaults to approved	To be verified
FT-02	Auth	Register a new seller account	Account is created with status = pending, dashboard restricted until approval	To be verified
FT-03	Auth	Login with valid credentials	User is authenticated and redirected per role	To be verified
FT-04	Auth	Login with invalid credentials	Login fails with a clear error message	To be verified
FT-05	Catalog	Search for 'saree' (English)	Products tagged শাড়ি are returned	To be verified

Test Case ID	Module	Test Scenario	Expected Result	Status
FT-06	Catalog	Add product exceeding available stock to cart	System blocks the addition / caps at available stock	To be verified
FT-07	Cart	Apply a valid, unexpired voucher above minimum purchase	Discount is applied and grand total recalculated	To be verified
FT-08	Cart	Apply an expired voucher	Voucher is rejected with an explanatory message	To be verified
FT-09	Checkout	Place an order with Cash on Delivery	Order is created with payment_status = pending, no transaction ID required	To be verified
FT-10	Checkout	Place an order against a frozen seller's product	Order is rejected before creation	To be verified
FT-11	Admin	Approve a pending seller	Seller status changes to approved; dashboard unlocks	To be verified
FT-12	Admin	Freeze a shop	Shop's products disappear from public listings immediately	To be verified
FT-13	Delivery	Update an order to 'Delivered'	Status change is visible on consumer order history and admin dashboard	To be verified
FT-14	Consumer	Download an invoice PDF for a completed order	A correctly itemized PDF is generated and downloadable	To be verified

7.5 Usability Testing

Usability was addressed at the design stage through the wireframing process shown in Chapter 6, and through the consistent, bilingual, form-based interaction pattern described in Section 4.8 — intended to keep the interface approachable for the platform's specific, non-technical rural seller audience. No dedicated usability testing session (e.g. moderated user testing with real sellers) was documented for this project.

7.6 Responsive Testing

Responsiveness is implemented through Tailwind CSS breakpoint utilities across all major pages and dashboards, as noted in Section 4.8. No formal cross-device test matrix (specific devices, screen sizes, or browser/OS combinations) was documented for this project; manual verification during development on desktop and mobile viewport sizes is the most that can be claimed on the available evidence.

7.7 Security Testing

The functional test cases in Section 7.4 include basic authentication and authorization checks (invalid login, order rejection for frozen shops). No dedicated penetration test, vulnerability scan, or formal security audit was documented for this project. The main security-relevant implementation gap identified during this review is the client-side exposure of the Gemini API key, discussed in Section 4.9.

7.8 Browser Compatibility

No formal browser compatibility test matrix was documented. Because Karumart uses standard, actively maintained libraries (React 19, Vite, Tailwind CSS 4) without any browser-specific code paths, current versions of major evergreen browsers (Chrome, Firefox, Edge, Safari) are the recommended minimum test targets for future compatibility verification.

7.9 Testing Summary

Karumart's testing evidence now includes a directly observed, first-hand terminal screenshot of an actual Vitest run (Figure 7.1): 10 test files (8 passing, 2 with failures) and 146 individual tests (143 passing, 3 failing — a 98.0% pass rate), captured from the real development environment rather than taken solely from documentation. This is a stronger evidentiary basis than was available previously, and it is reported here exactly as observed, including the discrepancy with the internal testing report's higher claimed test count. Beyond this automated layer, a formally scoped but not-yet-executed twenty-case integration test plan and a fourteen-case functional test table (Table 7.1) remain outstanding. Resolving the 3 known failing unit tests, and then executing the integration and functional test plans and recording their actual results, are identified as the priority next steps ahead of any production deployment.

Chapter 8: Results and Discussion

8.1 Project Outcome

The Karumart development effort produced a working, deployed, bilingual multi-vendor marketplace application covering the full range of features described in Chapter 5: consumer registration, browsing, search, cart, checkout, order tracking, invoicing, reviews, and complaints; seller registration, product management with AI-assisted descriptions, and order fulfilment; delivery-personnel order management; and a thirteen-tab administrator control panel covering approval workflows, vouchers, banners, and platform statistics. The project's sprint records describe a second live deployment milestone ("2nd time Live Online Deployment") completed during Sprint 6, indicating the system reached a hosted, testable state before the end of development.

8.2 Achievement of Objectives

Table 8.1: Achievement of Objectives

Objective	Implementation / Outcome	Status
Role-based authentication for four user types	Implemented via Supabase Auth + profiles.role, enforced through ProtectedRoute and RLS	Achieved
Self-service seller dashboard with admin approval	Implemented in SellerDashboard.tsx; sellers default to 'pending' status	Achieved
Multilingual browse/search/purchase for consumers	Implemented via searchUtils.ts, translations.ts, Home.tsx, Cart.tsx, Checkout.tsx	Achieved
Full order lifecycle including returns	Implemented via the OrderStatus type and status-transition UI across dashboards	Achieved
Admin tools for approval, freezing, vouchers, and monitoring	Implemented in the 13-tab AdminDashboard.tsx	Achieved
AI-assisted seller product descriptions	Implemented via geminiService.generateProductDescription()	Achieved
AI-powered customer support chatbot	Implemented via AIChatbot.tsx + geminiService.chatWithSupport()	Achieved
District-aware delivery dashboard with real-time status	Implemented in DeliveryDashboard.tsx with district filtering	Achieved
Live third-party payment gateway integration	Payment methods exist in the UI, but bKash/Nagad/SSLCommerz responses are simulated, not live	Partially achieved

Objective	Implementation / Outcome	Status
Comprehensive, executed test coverage	Unit/context suite executed with evidence (146 tests, 143 passing, 98.0% pass rate, 3 known failures); integration plan defined but not yet executed	Partially achieved

8.3 Benefits of Karumart

- Direct market access for rural artisans without the margin loss associated with intermediaries.
- A moderated, trust-signalling environment (approval status, badges, ratings) for consumers purchasing from previously unfamiliar sellers.
- A single coordinated system connecting product listing, ordering, and last-mile delivery, replacing what was previously an ad hoc, manually coordinated process.
- Reduced onboarding friction for non-technical sellers through a guided dashboard and AI-assisted description writing.
- Operational oversight for the platform owner through the admin approval workflow, shop-freeze mechanism, and audit log.

8.4 Challenges

Several challenges can reasonably be inferred from the project's own sprint records. Tasks such as "Implement product search and category filtering," "Prepare system testing report and test case documentation," and various dashboard integration items recur across multiple sprints as carry-over (incomplete) items before being marked complete in a later sprint, indicating that estimation for search/filtering functionality and for testing documentation was optimistic relative to actual effort required. The breadth of the admin dashboard (thirteen functional tabs, over 3,700 lines of a single component file) also represents a substantial integration effort concentrated in a small number of sprints (Sprint 6 and Sprint 7 in particular), which is consistent with a a small team building a feature-rich administrative panel under semester time constraints.

8.5 Current Limitations

- Payment gateway integrations (bKash, Nagad, SSLCommerz) are simulated rather than connected to live payment processors.
- The Gemini API key is exposed to the client bundle rather than proxied through a private backend.
- Stock decrement at checkout is not wrapped in a single atomic database transaction, creating a theoretical race condition under concurrent orders for the same low-stock item.
- The formally scoped integration test plan has not yet been executed and recorded.
- No native mobile application exists; the platform is web-only, albeit responsive.

8.6 Discussion

Measured against the problem statement in Chapter 1 — the lack of an accessible, trustworthy digital channel connecting rural artisans to buyers — Karumart's implemented feature set directly addresses each stated concern: a low-friction, guided seller onboarding process; an admin-moderated approval

workflow that gives buyers a basis for trust; bilingual search and browsing suited to the platform's target users; and an integrated order-to-delivery pipeline that replaces ad hoc coordination. The most significant gap between the system's current state and a production-ready deployment is the payment layer: because digital payment methods are simulated, the platform's real-world usefulness for cashless transactions is not yet fully realized, even though the checkout, order, and invoicing logic built around those payment methods is otherwise complete. Closing this gap, together with executing the documented integration and functional test plans, represents the clearest path from the current academic prototype to a deployable commercial platform.

Chapter 9: Conclusion and Future Work

9.1 Conclusion

This report has documented the design and implementation of Karumart, a bilingual, role-based, multi-vendor e-commerce platform built to connect rural Bangladeshi artisans and craftspeople with a broader base of consumers. Beginning from a genuine client brief describing the specific difficulties faced by rural sellers of handloom textiles, pottery, and other traditional goods, the project team (Group 16, Team Pi) translated this need into a concrete requirement specification and, across roughly ten two-week sprints, implemented a working platform built on React, TypeScript, and Supabase.

The resulting system supports four distinct user roles — Consumer, Seller, Delivery Man, and Admin — each served by a dedicated dashboard, and implements the full commercial lifecycle of the marketplace: seller onboarding subject to administrator approval, product listing with automatic SKU generation and moderation, bilingual product search and discovery, a persistent cart with voucher support, a checkout flow that validates stock and seller status before committing an order, a defined delivery-status lifecycle managed through a dedicated delivery dashboard, and post-purchase features including PDF invoicing, ratings and reviews, and a complaint/return workflow. Two AI-powered features — automatic Bengali product-description generation and a context-aware shopping assistant chatbot — are integrated using Google's Gemini API, extending the platform beyond a conventional CRUD-based storefront.

At the same time, this report has been careful to represent the system's current limitations accurately rather than overstating its readiness: digital payment methods are currently simulated rather than live, the formally scoped integration test plan remains to be executed, and a small number of implementation choices (client-exposed AI API key, non-atomic stock decrement) would benefit from hardening before a production launch. Taken as a whole, Karumart demonstrates a complete, coherently architected solution to the problem set out by its client, and provides a solid foundation for the enhancements outlined below.

9.2 Future Enhancements

The following items are identified as realistic future work, building on the existing system rather than describing functionality already implemented:

- Live payment gateway integration: replace the simulated bKash, Nagad, and SSLCommerz flows in `paymentService.ts` with genuine server-mediated integrations to the corresponding payment providers' APIs.
- Backend-proxied AI calls: move Gemini API calls behind a private backend endpoint so that the API key is no longer exposed in client-side JavaScript.
- Atomic order transactions: move order creation, order-item insertion, and stock decrement into a single Supabase database function (stored procedure) or transaction to remove the current race-condition risk under concurrent checkouts.
- Resolve known test failures and complete documented testing: fix the 3 currently failing unit tests identified in Figure 7.1, then execute the twenty-case integration test plan and the functional test cases identified in Chapter 7, recording actual results in place of the current "to be verified" placeholders.

- Native or hybrid mobile application: extend access beyond the responsive web application to a dedicated mobile app, particularly valuable for delivery personnel working primarily on mobile devices.
- Advanced analytics and recommendations: extend the admin statistics dashboard toward seller-level analytics and consumer-facing product recommendations.
- Automated email/SMS notifications: notify consumers and sellers of order status changes rather than relying solely on in-app visibility.
- Formal email verification at registration: the client's original requirement documentation specifies OTP or link-based email verification at sign-up; implementing this against Supabase Auth's built-in email confirmation flow would close a currently open item from the original requirement set.

References:

- [1] React, "React Documentation," Meta Platforms, Inc. [Online]. Available: <https://react.dev/>
- [2] Vite, "Vite Documentation," VoidZero Inc. [Online]. Available: <https://vitejs.dev/>
- [3] TypeScript, "TypeScript Documentation," Microsoft Corporation. [Online]. Available: <https://www.typescriptlang.org/docs/>
- [4] Supabase, "Supabase Documentation," Supabase Inc. [Online]. Available: <https://supabase.com/docs>
- [5] PostgreSQL Global Development Group, "PostgreSQL Documentation." [Online]. Available: <https://www.postgresql.org/docs/>
- [6] Tailwind Labs, "Tailwind CSS Documentation." [Online]. Available: <https://tailwindcss.com/docs>
- [7] shadcn, "shadcn/ui Documentation." [Online]. Available: <https://ui.shadcn.com/>
- [8] Google, "Gemini API Documentation," Google LLC. [Online]. Available: <https://ai.google.dev/gemini-api/docs>
- [9] React Router, "React Router Documentation," Remix Software, Inc. [Online]. Available: <https://reactrouter.com/>
- [10] Express, "Express Documentation," OpenJS Foundation. [Online]. Available: <https://expressjs.com/>
- [11] Karumart Group 16 Project Team, "Project Requirement & User Story Document (Karumart)," East West University, CSE412 course document, 2026.
- [12] Karumart Group 16 Project Team, "KaruMart Project Report," East West University, CSE412 course document, 2026.
- [13] Karumart client-provided requirement document, "Karumart — Project Overview and Feature Modules," supplied by Mr. Najmul Hasan Naim, 2026.

Appendix A: Important Code Snippets

Selected code excerpts illustrating key business logic are reproduced in Sections 5.9 and 5.10 of this report, including the shopping-cart total calculation (`CartContext.tsx`), the unified payment dispatcher (`paymentService.ts`), and the checkout stock/shop validation query (`Checkout.tsx`). These excerpts are drawn directly from the project's source files and are not reproduced again in full here to avoid duplication.

Appendix B: Database Schema

The complete baseline schema is defined in the project's `database_setup.sql` script, covering the profiles, products, orders, order_items, banners, static_banners, vouchers, complaints, admin_logs, and site_settings tables, together with their Row Level Security policies. The entity summary and relationship diagram are presented in Sections 4.6 and 4.7 of this report.

Appendix C: Additional Design Artefacts

Beyond the six wireframe pages presented in Chapter 6, the project's original "Overall Architecture" design document also includes annotated callouts (numbered labels identifying each UI region, e.g. "Sidebar Toggle," "Summary Cards," "Row Actions") that were used internally by the design team to specify component behaviour to the development team. These annotations are visible in Figures 6.3 and 6.4 and were used as the basis for the corresponding React component structure described in Chapter 5.

Appendix D: Test Cases

The full functional test case table is presented in Table 7.1 (Section 7.4) of this report. The complete twenty-case integration test plan (IT-01 through IT-20), covering authentication, cart/checkout, payment routing, search, multilingual switching, admin operations, dashboard access, and voucher/discount calculation, is documented in the project's internal Integration Testing Report and summarized in Section 7.2; as noted there, its status column is recorded as "To Be Verified" pending an executed test run.